

Engineering NEA 2026

Albin Prince Ashok Arockia Doss





Problem Analysis



Description of the problem

The problem I am trying to solve is to help children to count. Counting can be a challenging skill for children to learn. My task is to engineer a device or system that will support children with learning how to count.

Brief

I am going to design and build a toy to teach children how to count. I am building a toy to make the children interact and engage in a fun way. I want to be based around shopping . It could have different modes for the parent's supervision and children on their own.

What I need to find out

Size : how big will it be to fit all the necessary components for it to function

Power : how will the toy be powered is it by plug adapter on the wall or by battery

Components : will the power be sufficient to power all the different components and where all the components will fit. Are they power efficient



Relevant Factor

- **Design** - Age-appropriate design such as large buttons, safe materials and simple User Interface.
- **Safety factors** - such as low voltage components and debounced buttons to avoid false triggers caused by children. As well as no sharp or hazardous materials which can cause harm to humans.
- **Durability** - It must be durable to withstand repeated button presses and rough handling by children.
- **Cost** – It must be relatively cheap to produce so no expensive electronics
- **Weight** – It should be light enough for an adult to lift and move easily, but heavy enough that children cannot lift it and injure themselves.
- **Reliability** – The till must be able to run for a long as it is plugged in as it is power by a wall adapter.
- **User interaction** – does the till achieve it purpose of helping children count while making the experience fun
- **Ergonomics** – how easy is it for the user to play with and is it easy to setup

Existing Product Research

1.Simple Counting Buttons

- A basic panel of buttons labelled 1–10.
- Pros: Simple, safe.
- Cons: Limited engagement as it does not simulate real-world counting scenarios.



2.Digital Tablet

- Interactive tablet for counting items and selecting the number of item in the shopping cart.
- Pros: Highly interactive, relatively easy to build
- Cons: Less tactile, expensive, requires supervision, not child-proof.



3.Interactive Shopping Till Toy

- A cash register-style toy where children scan items, press buttons, and count totals.
- Pros: Fun, hands-on learning, supports multiple play modes (parent-supervised or independent), engages children in real-world counting scenarios.
- Cons: Slightly more complex to build



Bill of Material

Components	Price	Sourced from
Df Player Mini	£1.54	Amazon
Resistor	£1.70	Temu
Flyback Diode	£1.04	Temu
Wire Splitter	£1.73	Temu
Power supply	£5.04	Temu
Barrel Jack Converter	£2.46	Temu
Barrel Jack power cable	£0.98	Temu
Buck Converter	£1.87	Temu
RFID module and kit	£1.89	Temu
Springs	£0.97	Temu
Rail Sliders	£2.76	Temu
Buttons	£4.08	Temu
OLED	£1.74	Temu
Mega	£5.36	Temu
Transistors	£9.99	Temu
Speaker	£1.46	Temu

Specifications

Microcontroller	ATmega2560
Operating Voltage	5V
Input Voltage	7–12V
Input Voltage (limit)	6–20V
Digital I/O Pins	54
Analog Input Pins	16
DC Current for I/O Pin	20 mA
DC Current for 3.3V Pin	50 mA
Flash Memory	256 KB
SRAM	8KB
EEPROM	4KB
Clock Speed	16MHz
USB Connection	Type B
Dimensions	101.52 mm × 53.3 mm
Weight	37 g

Pinout Explanations for My Project

- Digital Pins (D0–D53): Used for sensors, buttons, LEDs, and actuators.
- Analog Pins (A0–A15): Measure variable signals from sensors. These values are converted to digital data for processing.
- Serial Ports (0–3): Allow the Mega to communicate with multiple modules
- VIN: Input voltage from an external power source.
- 5V and 3.3V: Output regulated voltages for modules.
- GND: Ground connection (must be common across all components).
- SPI Pins (50–53): Used for fast communication with modules like RFID or SD cards.
- I2C Pins (20, 21): Used to connect devices like displays or sensors using only two wires.

Arduino Mega



What is it

The Arduino Mega 2560 is an open-source microcontroller board built around the 8-bit ATmega2560, designed for complex electronics projects requiring more I/O and memory than boards like the Arduino Uno. It is programmed using the free Arduino Software (IDE) supporting C and C++, with code uploaded via USB. The board is suitable for advanced robotics, 3D printers, home automation, and data logging. More information is available on the Arduino website.

Pinouts:

- Digital Pins: D0–D53
- PWM Pins: D2–D13, D44–D46
- Analog Pins: A0–A15
- Communication:
 - UARTs (Serial Ports): 4 (Serial0 to Serial3)
 - I2C: SDA (20), SCL (21)
 - SPI: 50 (MISO), 51 (MOSI), 52 (SCK), 53 (SS)

Why this system is good for my project

The Arduino Mega 2560 is perfect for projects that need many sensors, outputs, or communication modules. It has a large number of input/output pins, which allows me to connect multiple components like sensors, modules, LEDs, and displays all at once. Because of its greater memory and processing power, the Mega can handle more complex programs than smaller boards like the Uno. This makes it excellent for my system as it requires a large memory due to extensive code and multiple libraries for all the different components I am using. It's still beginner-friendly, supports Arduino IDE, and has a wide community for troubleshooting and guidance.

Advantages

1. More I/O Pins: 54 digital and 16 analogue pins allow many modules and sensors to be connected simultaneously.
2. High Memory Capacity: 256 KB flash memory lets me run large, complex programs.
3. Multiple Communication Channels: 4 serial ports (UARTs), I2C, and SPI — great for multitasking
4. Beginner Friendly: Compatible with the Arduino IDE; large online support community.
5. Stable Power Supply: Can be powered via USB or an external source, suitable for long-term projects.
6. Open Source: Circuit diagrams and libraries are freely available for learning and customization.

Disadvantages

1. Larger Size: The Mega is physically bigger than the Uno, making it harder to fit into compact projects.
2. Higher Cost: More expensive than smaller boards, though it is still affordable
3. Power Consumption: Uses more energy, so it's less suitable for battery-only systems.
4. Requires More Wiring: With many pins available, wiring can become complex if not organized carefully.

What is it ?

The DFPlayer Mini is a compact, low-cost MP3/WAV player module that can play audio files directly from a microSD card. It features a built-in DAC (Digital-to-Analog Converter) that allows connection to a small speaker without extra circuitry. It can be controlled either via UART serial communication or through digital I/O pins, making it very flexible for beginner and intermediate projects.

It's designed to simplify audio integration in microcontroller projects, removing the need for complex audio programming or expensive shields.

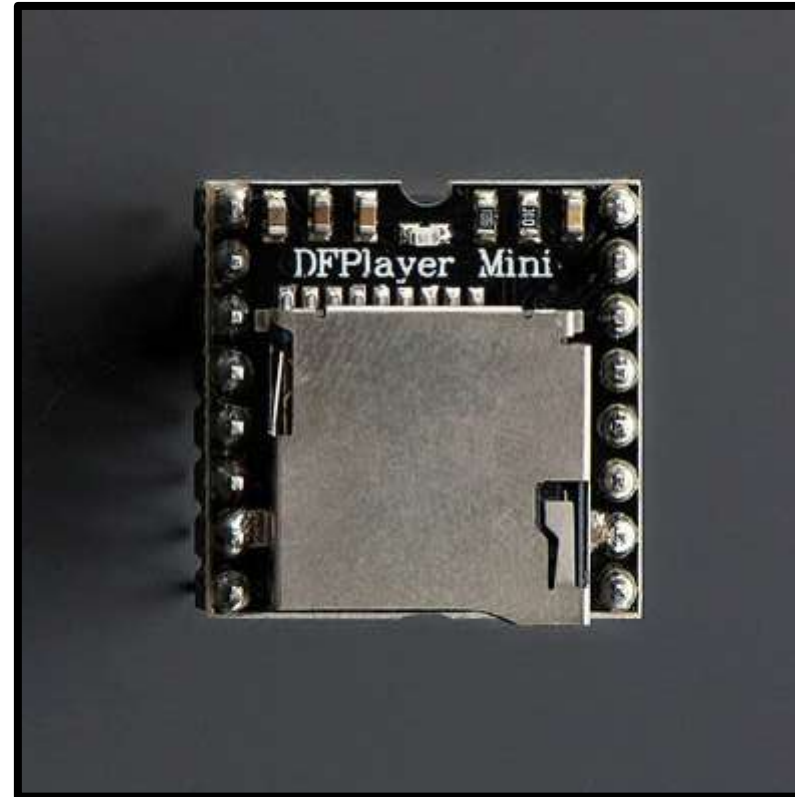
Why it's good for my system:

In my project, the DFPlayer Mini allows easy integration of sound and audio feedback. It is ideal for toy or educational projects because it enables instant sound playback, enhancing interactivity and engagement. The module is small, lightweight, and supports MP3/WAV formats, making it perfect for student-level projects and GCSE engineering assignments where audio cues or notifications improve user experience.

Pinout explanations for my project

- VCC: Power supply (3.2–5V) for the module.
- GND: Ground connection, completes the circuit.
- TX: Transmit pin; sends data from the DFPlayer Mini to the microcontroller.
- RX: Receive pin; receives serial commands from the microcontroller.
- SPK_1 / SPK_2: Direct connection to a speaker (8Ω recommended).
- IO_1 / IO_2: Optional digital pins for trigger/play functions without using serial commands.
- BUSY: Output pin that goes HIGH when the module is playing audio; can be used for status feedback.

DFPlayerMini

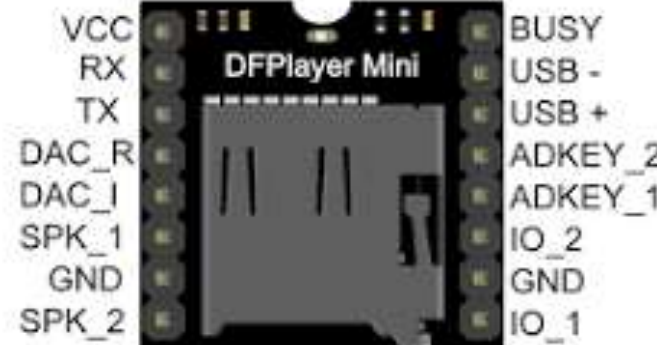


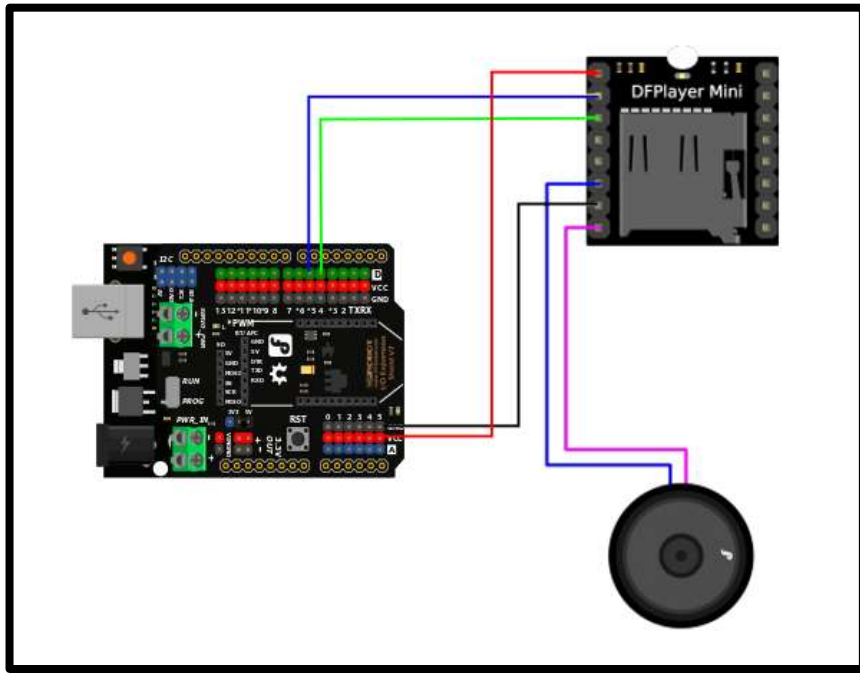
Advantages

1. The DFPlayer Mini is easy to use because it can play audio directly from a microSD card with minimal programming.
2. It is compact and lightweight, allowing it to fit inside small projects like toys or robots.
3. It provides direct audio output through its built-in DAC, so no external amplifier is needed.
4. It offers multiple control options, including UART for full playback and I/O pins for simple triggers.
5. It supports MP3 and WAV files, giving flexibility between storage efficiency and sound quality.
6. The module is affordable, making it suitable for student and hobby projects.
7. The BUSY pin allows feedback for synchronizing audio with other components.

Disadvantages

1. The DFPlayer Mini only supports MP3 and WAV files, so other audio formats cannot be used.
2. It only works with microSD cards up to 32GB and requires FAT16 or FAT32 formatting.
3. The speaker output is limited to small 8Ω speakers, so larger or louder speakers need an external amplifier.
4. It is sensitive to voltage, operating between 3.2–5V and can be damaged by higher voltages.
5. The module cannot stream audio or apply effects, so advanced features are limited.
6. Full control of volume, track, and folder selection requires microcontroller programming via UART.





`void setup()` : Runs once at the start. Initialises SPI, the RFID module, and serial communication.

`void loop()` : Runs repeatedly. Checks if a new RFID card is present and prints "Card detected" when detected.

`rfid.PCD_Init()` : Initialises the RFID reader to prepare it for detecting cards reliably.

`rfid.PICC_IsNewCardPresent()` : Detects if a new RFID card is near the reader.

`Serial.println()` : Sends a message to the serial monitor to confirm a card has been read.

Sample code

```
#include <MFRC522.h>
#define SS_PIN 10
#define RST_PIN 9
MFRC522 rfid(SS_PIN, RST_PIN);

void setup() {
  Serial.begin(9600);
  SPI.begin();
  rfid.PCD_Init(); // initialize RFID
}

void loop() {
  if (rfid.PICC_IsNewCardPresent()) {
    Serial.println("Card detected");
  }
}
```

Pinouts

- SDA to D10 : Selects the module for SPI communication
- SCK to D13 : SPI clock, synchronises data transfer
- MOSI to D11 : Sends data from Arduino to RFID
- MISO to D12 : Sends data from RFID to Arduino
- RST to D9 : Resets the module
- GND to GND : Ground connection
- 3.3V to 3.3V : Power supply for the module

What is it?

A Waveshare RGB LCD 1602 is a type of character display module that can show two lines of 16 characters each (hence “1602”). Unlike standard monochrome LCDs, this version includes an RGB backlight, allowing the screen to display a wide range of colors by mixing red, green, and blue light. It's commonly used in microcontroller projects to display text, sensor readings, or status messages, and can be controlled using interfaces like I²C or GPIO pins.

Why it's good for my system

In a project like a toy till or a counting game, the Waveshare RGB LCD 1602 acts as the Human-Machine Interface (HMI) — it provides clear, real-time feedback to the user. Its main advantages for this type of project are:

Simple & Readable Text Display: The 16×2 character layout is perfect for showing key information such as the current count, player score, or short messages like “Correct!” or “Try Again!”

Color Feedback with RGB Backlight: The adjustable RGB backlight can change color to represent different states — for example, green for correct answers or red for errors — adding visual appeal and intuitive feedback.

Low Power Usage: LCD technology is efficient, making it suitable for battery-powered devices or portable prototypes.

Durable & Easy to Integrate: The module is compact and robust, fitting easily into small enclosures or toy housings.

Good Visibility: The bright backlight ensures that text remains visible in various lighting conditions, indoors or outdoors.

LCD



Advantages

- 1.Low Power Consumption: Consumes very little current, making it energy-efficient and suitable for battery-powered projects like the toy till.
- 2.Compact Size: The small physical dimensions allow easy integration into compact designs without taking up much space.
- 3.Sharp and Clear Text Display: Provides high contrast and readable text and numbers, ensuring human-machine interaction is effective and clear.
- 4.Fast Refresh Rate: Updates content quickly, which is useful for displaying counters, progress, or changing messages in real time.
- 5.I2C Communication: Only requires two data pins (SDA and SCL), freeing up Arduino pins for other components such as buttons, RFID, or solenoids.

Disadvantages

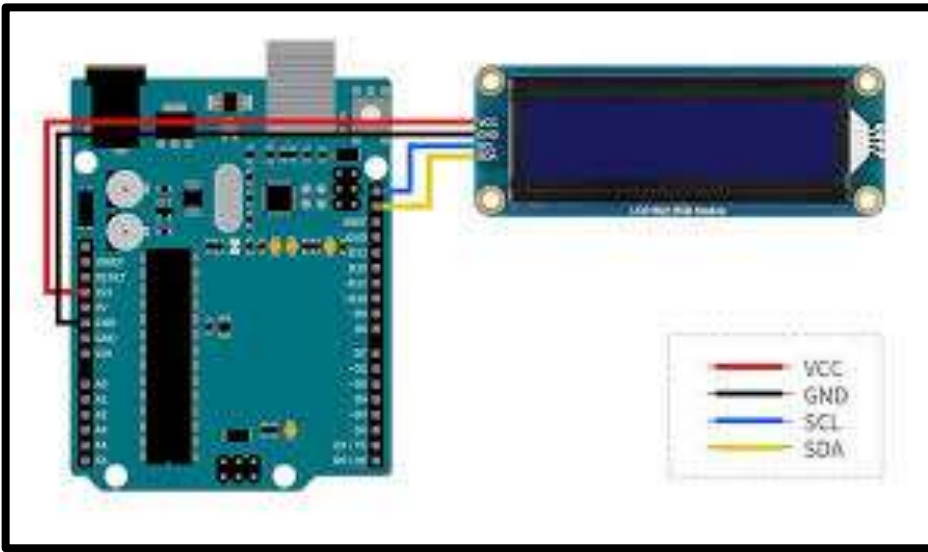
- 1.Monochrome Only: Can only display one colour (usually white), limiting visual design options compared to colour displays.
- 2.Limited Display Area: Small screen size restricts the amount of text or graphics that can be shown at once, which can be challenging for long messages.
- 3.Vulnerability to Damage: OLEDs can be sensitive to physical impact and static electricity, requiring careful handling during assembly.
- 4.Requires Library Support: Needs specific libraries and code to function, adding a dependency on software for correct operation.
- 5.Voltage Sensitivity: Typically operates at 3.3V–5V; incorrect voltage can damage the display.

Pinout explanation for my project

- GND : This is the ground connection, providing a common reference voltage for the display and completing the electrical circuit. Without a stable ground, the OLED cannot operate reliably.
- VCC : Supplies power to the OLED module. The 5V input ensures the internal electronics receive sufficient voltage for consistent operation and safe functionality.
- SCL : The Serial Clock Line of the I2C communication protocol. This pin synchronises the transfer of data between the Arduino and the OLED, ensuring signals are correctly timed and interpreted.
- SDA : The Serial Data Line of the I2C bus. This pin is used to send and receive data between the Arduino and the OLED, including text, numbers, or graphical information for the display.

Pinouts

- VCC → 3.3V or 5V
- GND → GND
- SDA → A4
- SCL → A5



Sample code

```
#include <Wire.h>
#include "Waveshare_LCD1602.h"
```

```
Waveshare_LCD1602 lcd(16,2);
int r,g,b,t=0;
void setup() {
    // initialize
    lcd.init();

    lcd.setCursor(0,0);
    lcd.send_string("Waveshare");
    lcd.setCursor(0,1);
    lcd.send_string("Hello,World!!!");
}

void loop() {
    delay(150);
}
```

<Wire.h> Enables I2C communication.
The LCD communicates via I2C, so the Arduino needs this library to send commands.

"Waveshare_LCD1602.h" → Provides functions to control the Waveshare 16x2 LCD.
Contains commands to initialize the LCD and send characters to it.

Waveshare_LCD1602 lcd(16, 2);
Creates an object for the 16x2 LCD so your program can control it. Without this, you couldn't send text or position the cursor.

lcd.init();
Initializes the LCD so it's ready to display text.

lcd.setCursor(0,0); / lcd.setCursor(0,1);
Positions the text at a specific row and column. Otherwise, text would start at a default location that might not be what you want.

lcd.send_string("Hello");
Sends characters to the LCD to actually display them. Without this, nothing appears.

loop() with delay()
Runs repeatedly. Currently only delays the loop to avoid flooding the LCD, but it could later update counters or other information dynamically.

Component Name	Image	Description	Advantages	Disadvantages
Arduino Mega		- The Arduino Mega is a microcontroller board based on the ATmega2560. It has 54 digital input/output pins (of which 14 can be used as PWM outputs), 16 analog inputs, 4 UARTs (hardware serial ports), a 16 MHz crystal oscillator, a USB connection, a power jack, an ICSP header, and a reset button.	- Its flexible and simple to get started with	- -Not cheap - Need c++ for it to work - And need the ide
RFID 522		- The MFRC522, often found in RC522 RFID modules, is a highly integrated, low-cost reader/writer IC by NXP for contactless communication at 13.56 MHz using the ISO/IEC 14443 A standard, including MIFARE and NTAG cards. It uses a 4-pin SPI connection for interfacing with microcontrollers and can communicate with RFID tags up to 5 cm away, with applications in access control and inventory.	- Low cost - Low power consumption - Versatile	- Limited read range - 3.3 v only - Susceptible to interference
Solenoids		- A solenoid is an electromagnet made of a cylindrical coil of wire that, when an electric current passes through it, generates a uniform and controllable magnetic field. In engineering, the term often refers to the electromechanical device that uses this magnetic field to convert electrical energy into linear or mechanical motion.	- Rapid response - Binary control - Remote operation	- - high energy consumption - Needs 12 volts
LCD 1602		- An LCD 1602 is a common, inexpensive, and versatile character-based Liquid Crystal Display (LCD) module used in various electronics projects, especially with microcontrollers like Arduino. The "1602" in its name signifies its display capacity of 16 columns and 2 rows, for a total of 32 characters.	- Low price - Easy to setup - Show custom characters	- Limited character capacity - Slow refresh rate

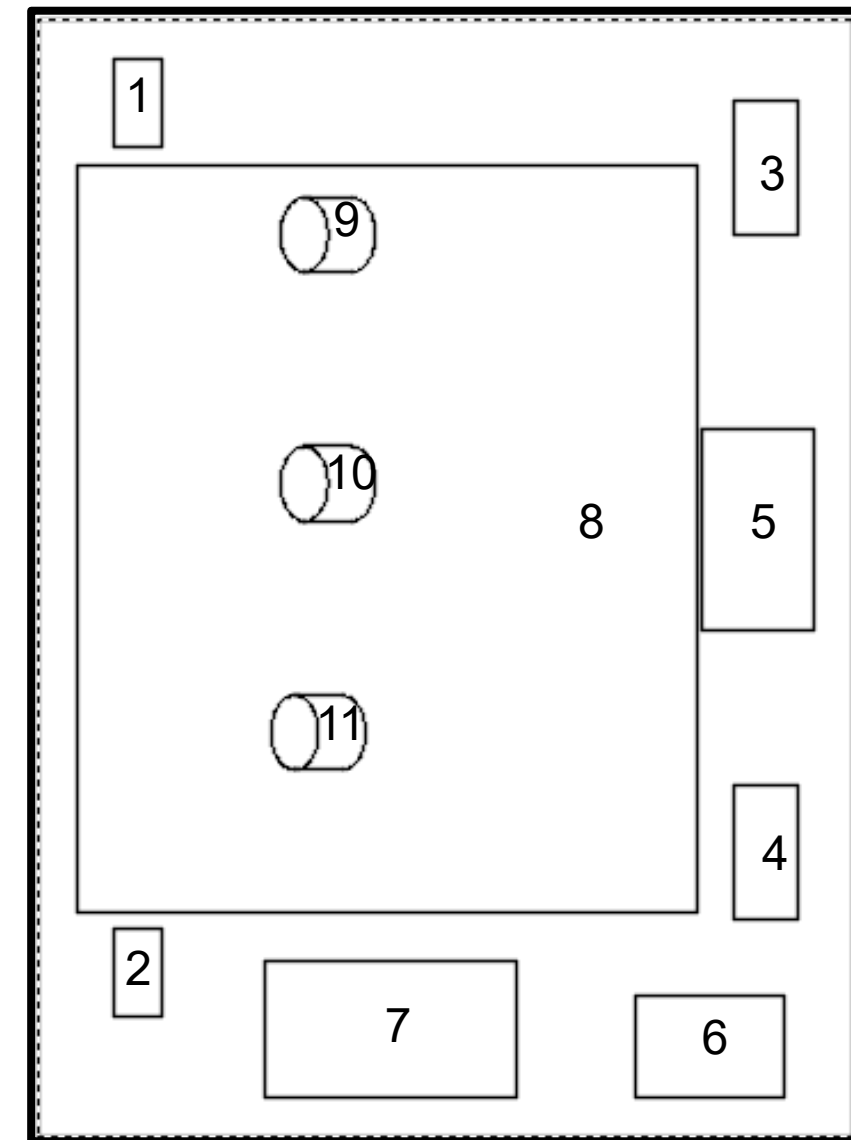
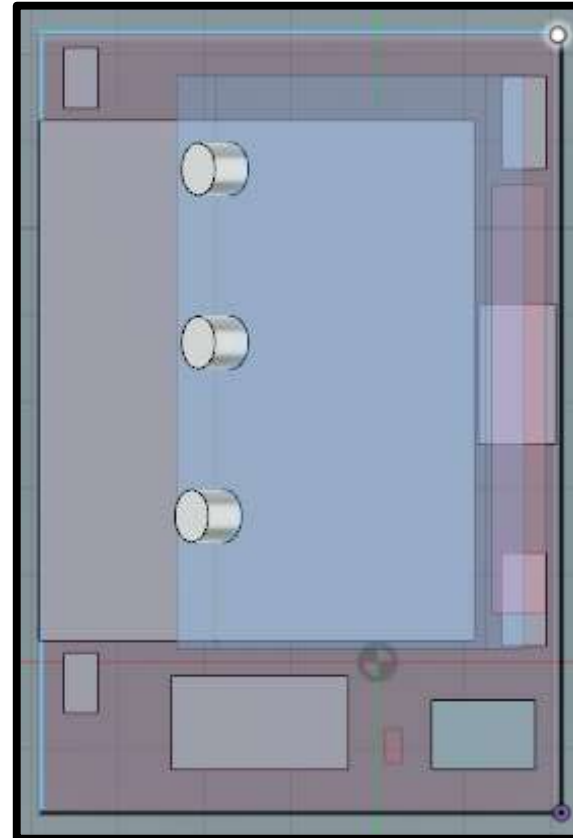
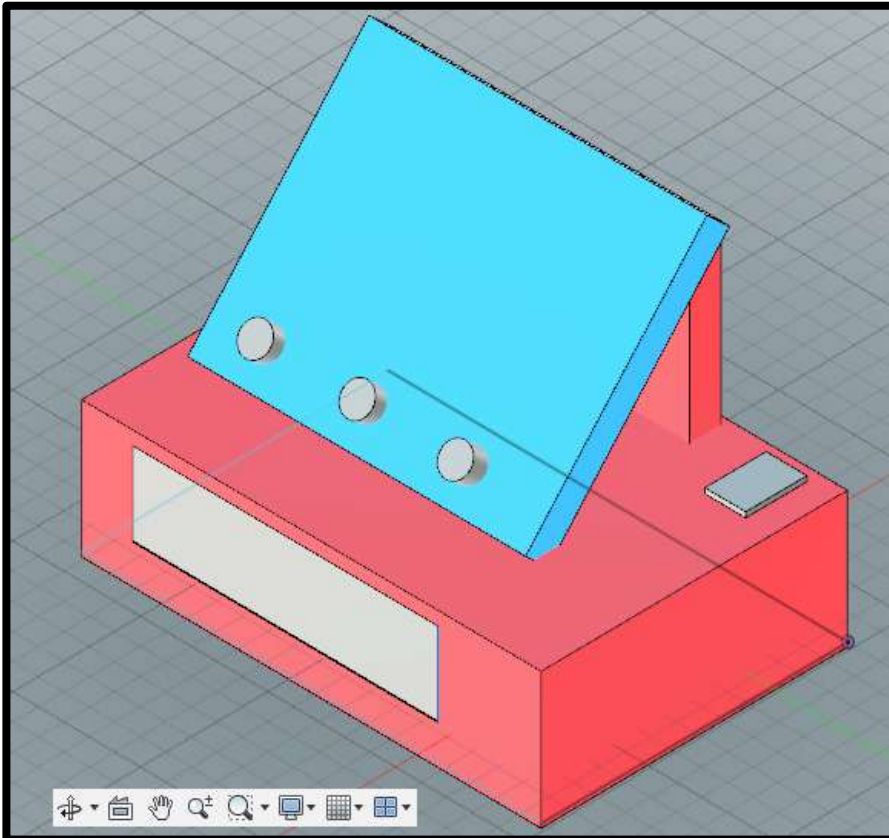
Initial Ideas



Idea 1

Overview of Idea:

This idea is a fun and educational toy till system designed to help children learn counting through interaction. The child is asked a question about quantities (for example, the number of fruits). They answer by pressing the counter button the correct number of times to represent their answer. Then they press the next button to submit. If the answer is correct, the till moves on to the next question; if not, the child tries again. Once all the questions have been answered correctly, the system plays a short tune through the speaker and unlocks the till drawer using a solenoid mechanism. The RFID module is used for shopkeeper login before the till can be used, ensuring it feels like a real interactive shop game. The Arduino Mega controls the logic, manages button presses, checks answers, and activates the solenoid through a control circuit.



- | | |
|---------------------------|-------------------|
| 1. Solenoid 1 | 7. Arduino Mega |
| 2. Solenoid 2 | 8. Till Draw |
| 3. Buck Converter 1 | 9. Counter Button |
| 4. Buck Converter 2 | 10. Next Button |
| 5. Solenoid Control Board | 11. Reset Button |
| 6. RFID Reader | |

Mechanisms:

The system includes two 12V solenoids that operate the till drawer lock. When the child completes all questions correctly, the Arduino activates a MOSFET solenoid control board, which connects power to the solenoid momentarily, retracting it and unlocking the drawer. The speaker plays a tune at the same time to celebrate success. Buttons are connected to digital inputs, and a reset button clears the counters and resets the game. The logic and sequence are all handled in the Arduino code.

Materials and Manufacturing Processes:

The outer casing will be made from 4mm laser-cut MDF for the main frame and 5mm MDF for the inner frame and till locking structure, chosen for durability, rigidity, and precision cutting. The component mount will also be laser-cut from MDF and fitted using PCB pillars and screws to hold the electronic boards (Arduino Mega, solenoid driver, RFID reader, DFPlayer Mini, and power converters) securely. Openings for the LCD screen, buttons, and speaker will be cut accurately using CAD/CAM software and laser-cutting technology. The mount will be attached to the frame using screws to allow for easy removal and maintenance.

Joints and Fixings:

The outer frame will be joined using PVA wood glue for strong permanent bonds between MDF panels. The removable access panel (to reach internal components) will be fixed using screws and nuts. The inner frame will use PCB standoffs and bolts to hold the electronic modules in position. The solenoid will be screwed directly into a 5mm MDF block, ensuring solid alignment with the locking mechanism of the drawer.

Advantages:

- Combines counting education with a fun, hands-on activity.
- Provides visual, auditory, and physical feedback (sound, screen, and drawer movement).
- Uses reliable components like Arduino and RFID for easy expansion and maintenance.
- Laser-cut precision gives a professional, safe, and sturdy finish.
- Modular structure allows for repair or upgrades without replacing the whole system.

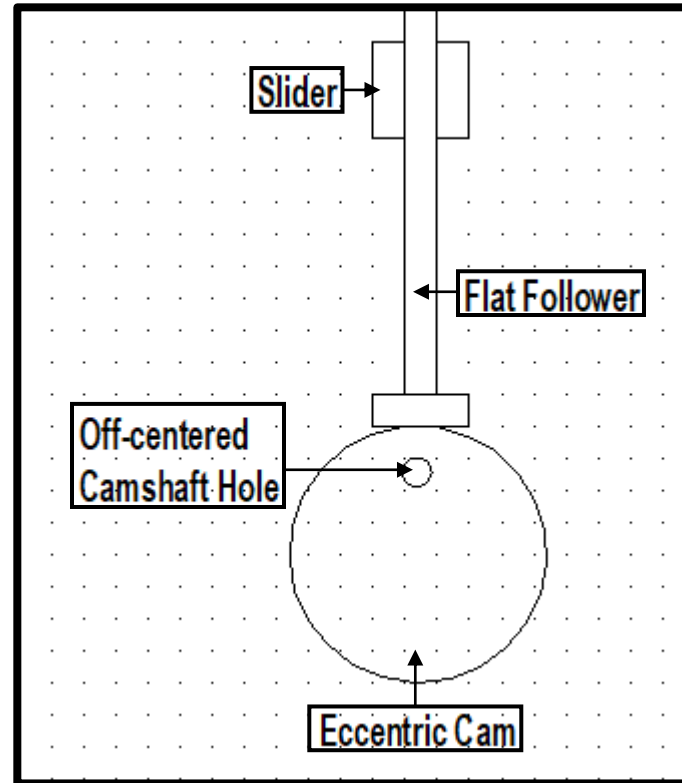
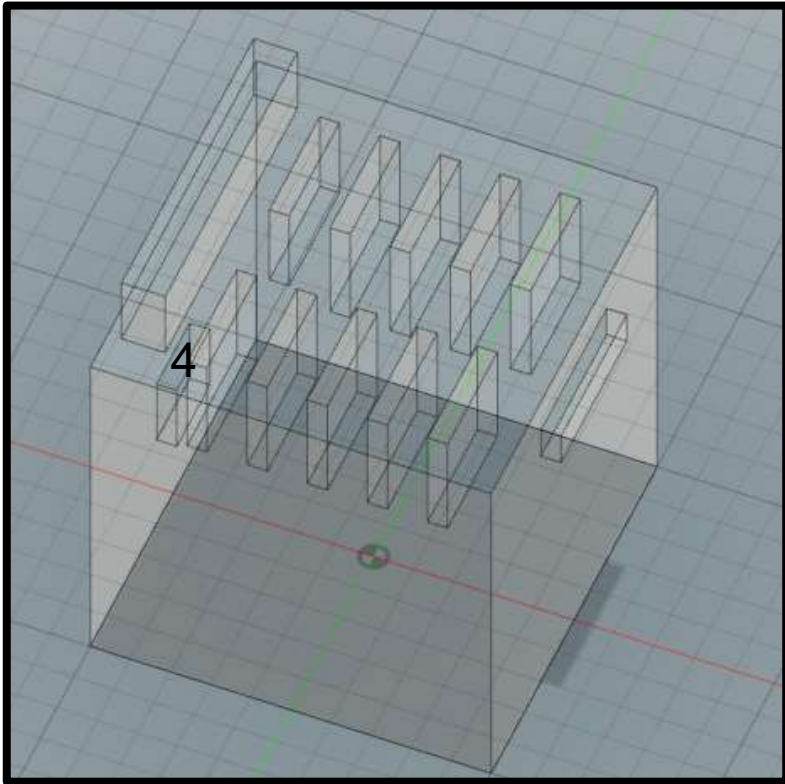
Disadvantages:

- Requires both 5V and 12V power, increasing wiring complexity.
- The solenoid system may consume more current and needs safe control via MOSFETs.
- If wiring is loose or incorrectly connected, the solenoids may fail to release properly.
- The system is more complex to assemble than simpler button-only counting toys.

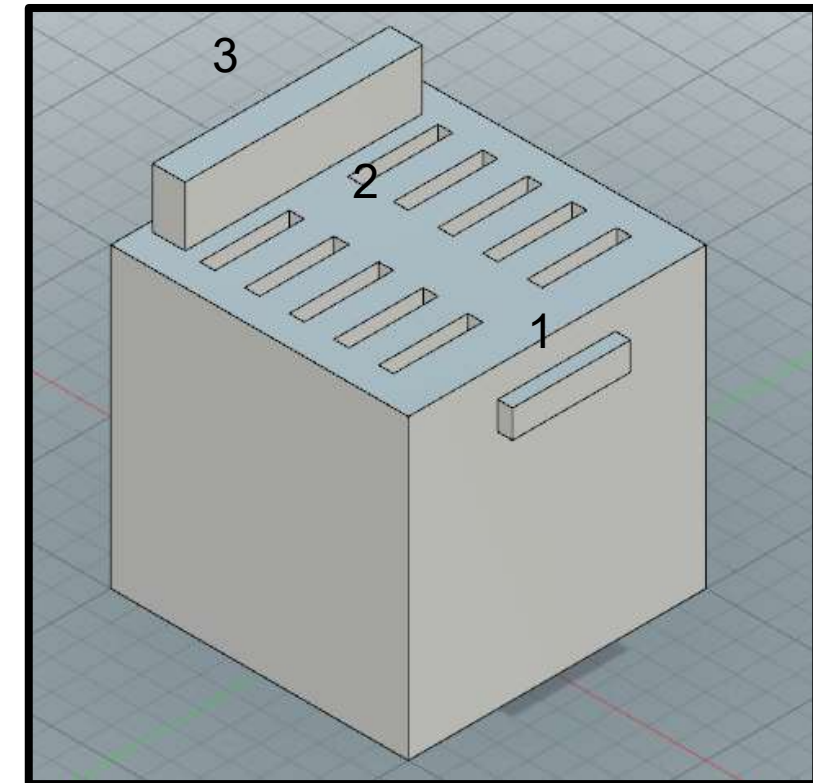
Idea 2

Overview of Idea:

This concept is an interactive educational till system designed to help children learn number sequencing. Each card has an RFID tag that stores a number. The child holds each card under the LCD screen, where an RFID reader hidden inside the casing detects and reads the tag. The Arduino Mega processes this data to check whether the cards are placed in the correct order. If all cards are scanned in sequence correctly, the system triggers a cam and follower mechanism that moves up and down, while the speaker plays a congratulatory tune. The LCD displays the child's progress and provides visual feedback, making the activity fun and educational.



1. LCD
2. Multiple RFID Slots
3. Cam Mechanism
4. Speaker



Mechanisms:

A cam and follower system is used to create movement when the correct sequence is achieved. The cam is attached to a small motor, and when triggered by the Arduino, it rotates to push the follower up and down, giving a simple but satisfying physical motion.

Materials and Manufacturing Processes:

The main casing is made from 4mm MDF, chosen for its strength, easy laser cutting, and low cost. The front display panel, where the LCD and RFID reader are mounted, is made from 3mm clear acrylic so that children can see lights or indicators through it. The cam components are 2D laser cut in MDF, as this material is lightweight and easy to produce accurately. Internal supports and fixing brackets can also be laser-cut from MDF to ensure the design is modular and easy to assemble.

Joints and Fixings:

The casing is assembled using PVA wood glue for permanent joints between MDF panels, while removable parts such as the back panel use small screws and bolts to allow maintenance access. The acrylic front is attached with M3 screws and nuts, which also prevent cracking and allow replacement if damaged. The electronic components are fixed with hot glue or mounting brackets to keep them stable inside the casing.

Advantages:

- Encourages active learning through play (visual, sound, and movement).
- Safe and durable construction with smooth MDF edges and acrylic protection.
- Uses simple, low-voltage electronics and easily programmable Arduino code.
- Modular design allows easy upgrades or repairs.
- The project combines STEM learning with fun interaction.

Disadvantages:

- Requires careful RFID calibration so each card reads correctly.
- Slightly more complex wiring compared to simpler button-based systems.
- laser cutting may take time if not all tools are available.
- If the RFID reader fails or misreads a tag, it can interrupt gameplay.

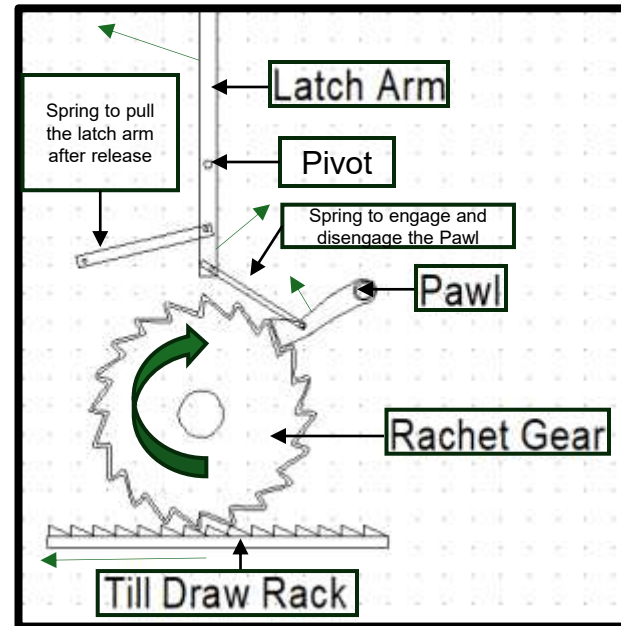
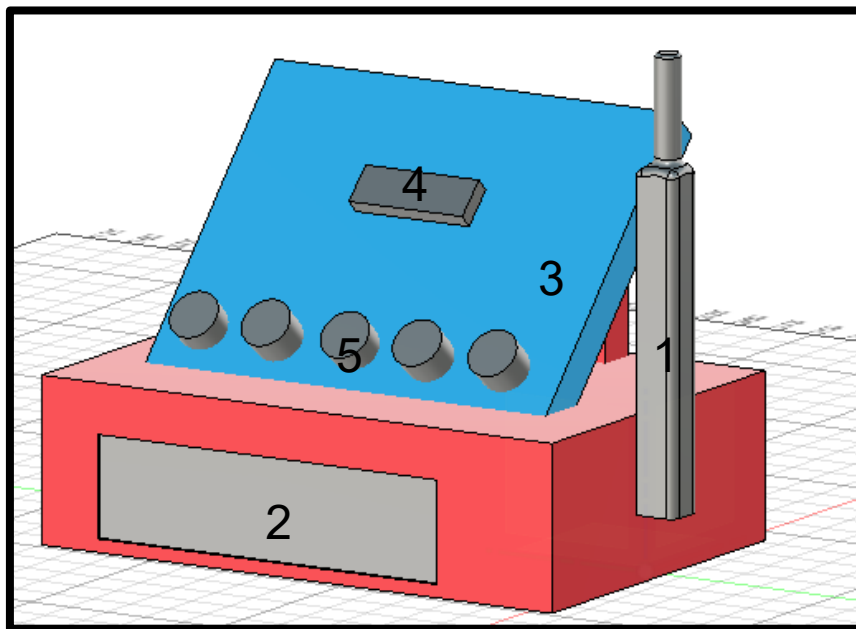
Idea 3

Overview of Idea

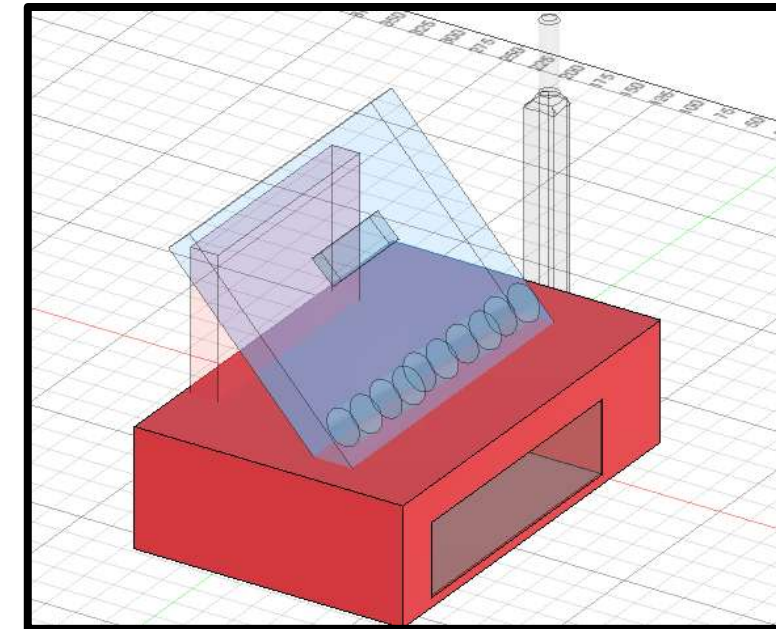
This concept is an interactive, educational cash register system inspired by traditional, old-fashioned mechanical registers rather than modern touchscreen designs. The system uses separate physical buttons for each input, encouraging tactile interaction and helping children understand numbers, sequencing, and basic arithmetic through hands-on play.

Instead of a large digital interface, numbers, messages, and totals are displayed on a small screen, such as a 16×2 I²C LCD display mounted on the top of the register, mimicking classic receipt displays.

The cash drawer release is purely mechanical—spring-loaded so it pops open when triggered, and released by a manual lever on the side, reinforcing cause-and-effect learning and mechanical understanding.



1. Mechanical Release Arm
2. Till draw
3. User Interface
4. LCD
5. Buttons



Mechanisms

The cash register uses a combination of simple electronic inputs and a mechanical drawer mechanism. Individual spring-return push buttons send signals to a microcontroller, which processes the input and displays numbers or messages on a 16×2 I²C LCD screen mounted on the top of the unit. The cash drawer is held closed by a mechanical latch and is released using a side-mounted lever that pivots to disengage the latch. Once released, a compression spring pushes the drawer forward, so it pops open smoothly, and when the drawer is pushed back in, the latch re-engages automatically.

Materials and Manufacturing Processes

The cash register will be manufactured using a combination of durable, low-cost materials and simple processes to ensure it is strong, safe, and suitable for repeated use. The main casing and drawer are made from MDF chosen for its strength, smooth finish, and ease of cutting and shaping. Panels are accurately cut using laser cutting, ensuring consistency and precise alignment, and then assembled using PVA wood glue and screws. The push buttons are manufactured from moulded plastic, providing durability and comfortable tactile feedback, and these are mounted through pre-drilled holes in the front panel. The cash drawer mechanism uses a steel compression spring and a small plastic latch; both are selected for their wear resistance and reliability. Electronic components, including the Arduino and 16×2 I²C LCD display, are fixed to internal mounting brackets using machine screws and spacers to prevent movement and protect wiring. Finally, the product is sanded smooth and finished with paint to improve appearance and make the surface easy to clean.

Joints and Fixings

The casing is assembled using PVA wood glue for permanent joints between MDF panels, while removable parts such as the back panel use small screws and bolts to allow maintenance access. The electronic components are fixed with hot glue or mounting brackets to keep them stable inside the casing.

Advantages

- Encourages tactile learning through physical buttons
- Mimics real-world, traditional cash registers
- Develops understanding of numbers, sequencing, and cause-effect
- Mechanical drawer adds realism without electronic complexity
- Simple electronics using low-cost, widely available components
- Durable and safe for classroom or home use

Disadvantages

- More mechanical parts compared to touchscreen alternatives
- Button wiring can become complex with many inputs
- Limited display size compared to larger screens
- Mechanical components may wear over time

Justification

For my NEA project, I have chosen to use Idea 1: A Toy Till.

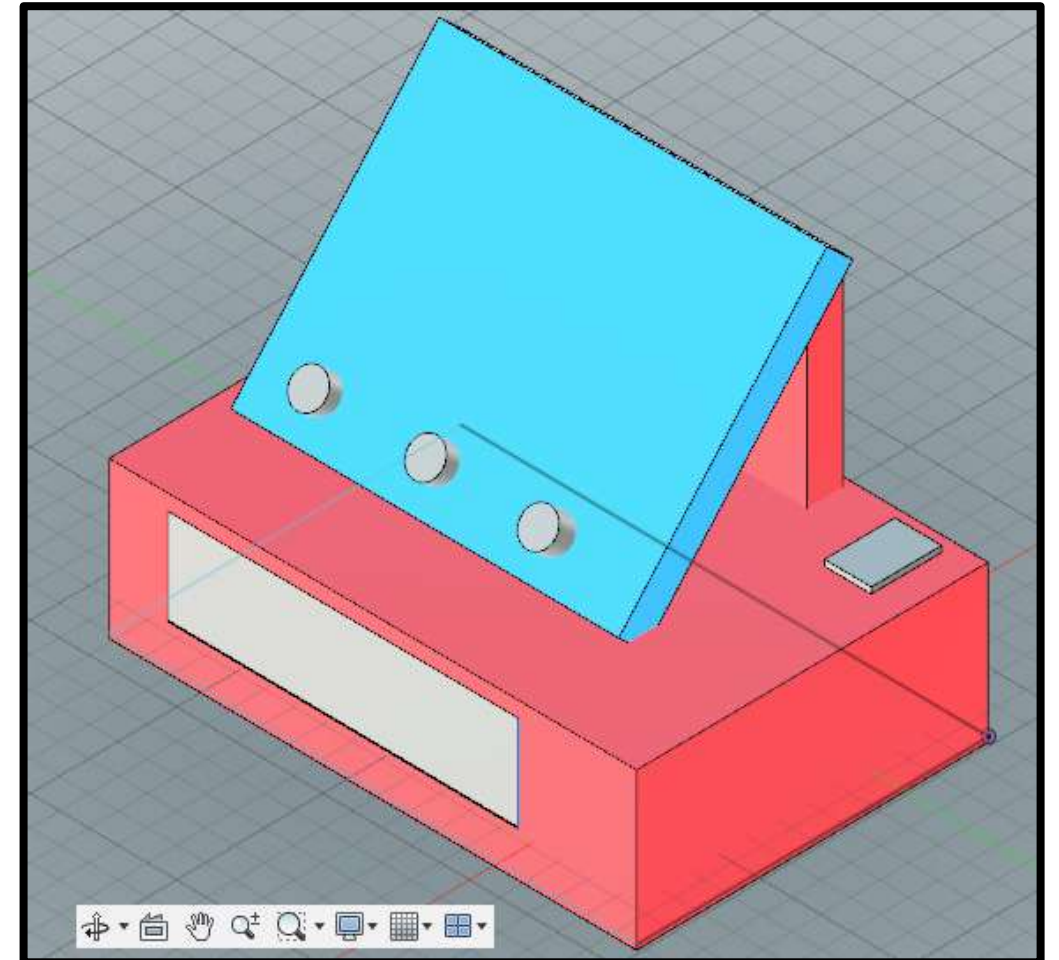
This product is designed to help young children improve their counting skills through a fun and engaging activity. The system asks the user questions based on quantities, in my product this will be counting fruits. The child responds by pressing a button multiple times to represent their answer. Once they have finished, they press a “next” button to submit it.

An Arduino Mega 2560 is used to control the system. It processes the inputs from the buttons and checks whether the answer is correct. If the answer is correct, the system moves on to the next question. If the answer is incorrect, the user is prompted to try again.

Once all questions have been answered correctly, the system rewards the user by playing a short tune through a speaker. It also activates a solenoid, which opens the till drawer, this means they can access the physical reward inside it such as a small toy or sweet. To make the system more interactive and realistic I have included an RFID module. This requires the child to “log in” as a shopkeeper before using the till, which simulates a real-life situation.

I chose this idea because it combines education and play, making learning more engaging for young users. It helps develop basic numeracy skills while also introducing simple technology concepts. The inclusion of rewards and realistic features, such as the opening drawer and RFID login, makes the product more motivating and interactive.

I had to change some aspects of the build to better suit my product. This included removing a 0.96” OLED to switch it out for a larger LCD, this means it is easier for the child to view the information shown on the screen. I also repositioned where the RFID reader was, so now it is moved to the User Interface.



Specification

Serial	Objective	Requirement	Justification	Testing Method
1	Buttons	Larger than 15mm	It must be easy to use by the child	Vernier Callipers / Ruler
2	Button Spacing	Wider than 20mm	It is to avoid accidental presses to 2 buttons at the same time	Vernier Callipers / Ruler
3	Display	Letter sizes equal to or larger than 3mm	It must be easy for the child to see	Vernier Callipers / Ruler
4	Casing Material Thickness	Thicker than 2.5mm	It must be durable	Vernier Callipers / Ruler
5	Audio Volume	Less than 70 decibels	It must be at a safe volume and not damage the child's ear	Volume Meter
6	Audio Frequency Range	80 Hz – 12,000 Hz	Must be audible to children while not damaging their auditory ability	Use a audio spectrum analyser
7	Counting Range	Must cover number range 1 to 10	To test and reinforce early counting skills	
8	Total Mass	Mass must be more than 1.5kg but less than 3kg	Since it is a toy till must be not easily moved by a child so they can't injure themselves	Weighing Scale
9	Force to open the draw	Must take less than 10N of force to open	It must be easy to open by a child	Force gauge measurement
10	Force to push the button	Must take less than 1N to push a button	It must be easily pressable by the child to avoid fatigue	Force gauge measurement
11	All Detachable Parts	Everything must be larger than 30mm by 30mm	It must no be a choking hazard	Vernier Callipers / Ruler / Eureka can
12	Start-up Time	Must take less than 5 seconds	It must be quick to load in order to maintain engagement and	Timer
13	User Interface	It must be larger than 44° but less than 54°	This is because the user need to be able to comfortably play with the toy by keeping it at an optimum angle	Protractor

Drawer

To ensure the till is durable and reliable for a child's use, I integrated metal telescopic rail sliders to manage the linear motion of the drawer. These sliders utilize steel ball bearings, which significantly reduce the coefficient of friction during operation. This ensures a smooth, low-effort movement while preventing the mechanical wear that would occur with a simple friction-fit wooden drawer. The high load-bearing capacity of the metal rails ensures the system can withstand repetitive use without sagging or jamming.

After laser cutting the chassis, I used a scribe and centre punch to accurately mark and indent the hole coordinates on the metal sliders. This preparation was vital to ensure the drill bit did not "slip and slide," allowing for perfectly aligned 3mm pilot holes. I opted for M3 machine screws and nuts rather than standard wood screws to create a high-strength, vibration-resistant connection. This mechanical fastening method is far more reliable for moving parts, as it prevents the hardware from loosening over time due to the repetitive kinetic energy of the drawer opening and closing, significantly increasing the longevity of the assembly.

For the final assembly of the chassis, I utilized finger joints (comb joints) to maximize the surface area for the PVA (Polyvinyl Acetate) wood glue. This interlocking geometry provides significant mechanical strength even before the adhesive sets. To further reinforce the structure, I added internal glue blocks at the corners. These blocks convert the joints into a reinforced "L-brace," significantly improving the torsional rigidity of the till base. This ensures the frame remains perfectly square and can withstand the repeated compressive stress and impact of the drawer being slammed shut without warping or splitting.

Metal Telescopic sliders

Finger Joints



Glue Blocks

M3 Screws

For Quality and Control check :
I performed a 'clearance test' to ensure the M3 nut heads did not interfere with the ball-bearing mechanism during full extension."

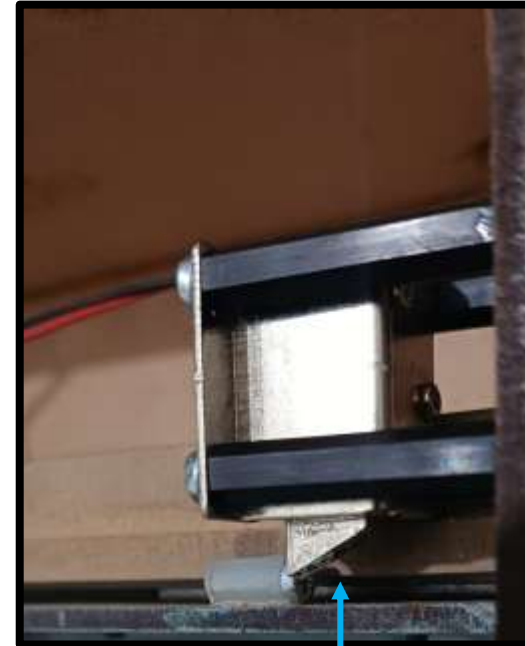
Locking System

To secure the till, I implemented a solenoid locking mechanism consisting of a plunger with a chamfered (triangular) tip and a custom-built latch. I specifically chose the chamfered profile to enable a 'slam-to-lock' feature. This geometry converts the linear kinetic force of the closing drawer into vertical displacement for the plunger. This is a more efficient user-centric design, as the drawer can be closed manually without needing to trigger the electronics, saving power and reducing wear on the solenoid coil.

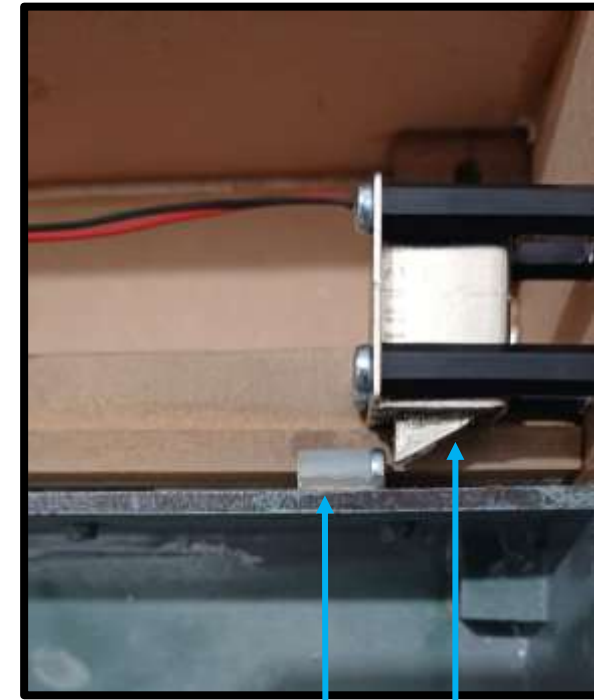
The latch was constructed using a PCB pillar and a hardened M3 steel screw. I chose steel for the contact point because it offers a higher wear resistance than the original plastic of the toy till. By using a metal-on-metal contact between the solenoid plunger and the M3 screw, it increases the longevity of the mechanism, ensuring the locking edge does not round off or fail after repeated use cycles.

When the drawer is locked and an external force is applied to pull it open, a high degree of shear force is exerted on the adhesive bond between the standoffs and the till's front panel. To counteract this, I used a high-strength surface adhesive with high tensile and shear properties. This ensures the stationary mounting remains rigid under stress, preventing the locking mechanism from being bypassed by manual force.

The internal compression spring of the solenoid provides two critical engineering functions in one: it enables an automatic reset, allowing the plunger to "snap" back and lock the drawer as soon as it is pushed shut, while also creating a 'Fail-Secure' system. This ensures that in the event of a circuit interruption, the spring tension keeps the plunger extended, keeping the till draw locked. This also means that when the toy is stored after being played with by the child, the till draw remains locked.

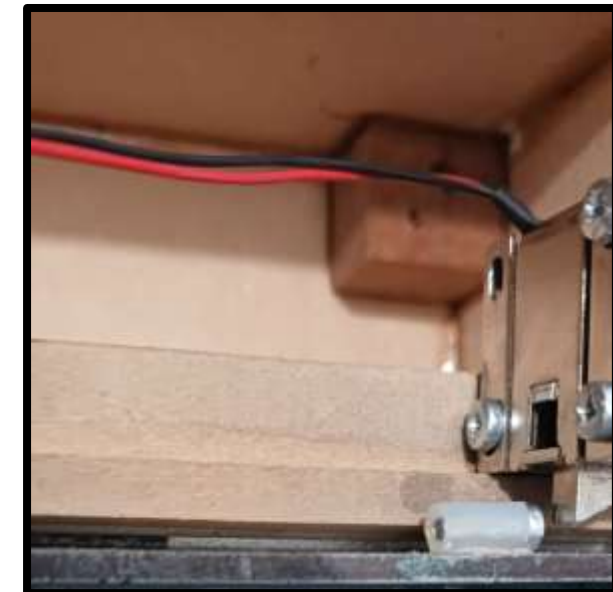


Chamfered Edge
(Triangular)



Latch

Plunger



Code Development

```
mirror_mod.mirror_object =  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True
```

```
selection at the end -add  
mirror_ob.select= 1  
mirror_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier.ob  
mirror_ob.select = 0  
= bpy.context.selected_object  
data.objects[one.name].select  
print("please select exactly 1")
```

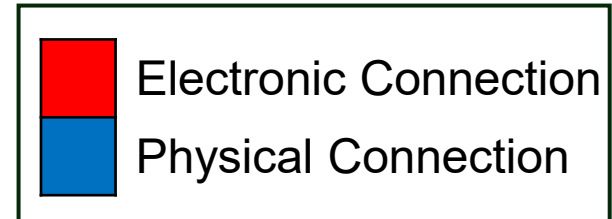
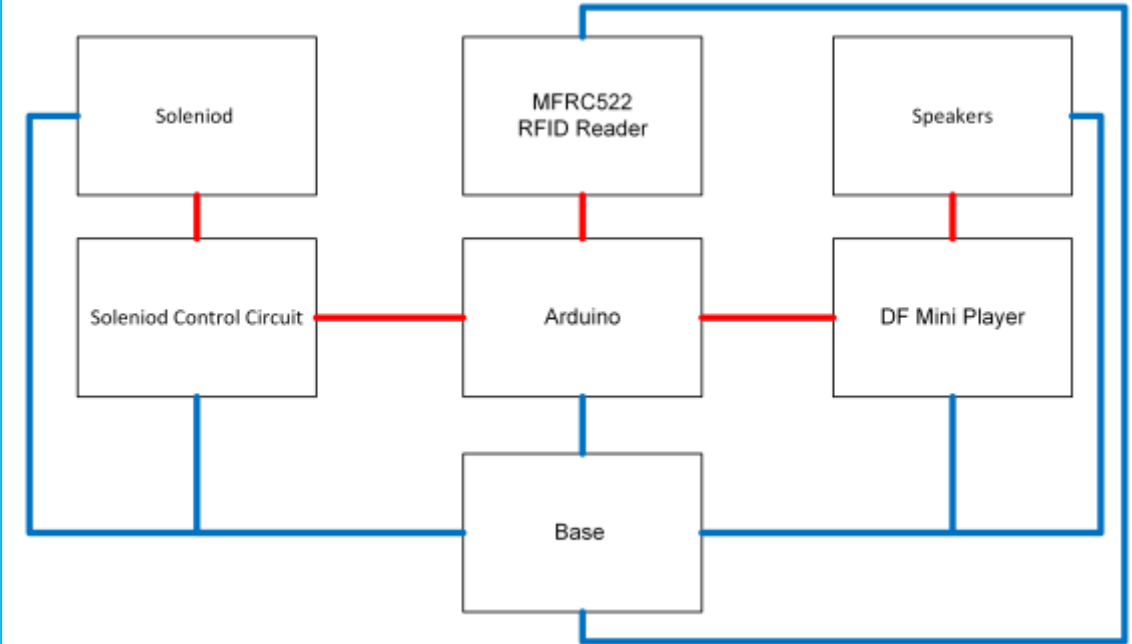
--- OPERATOR CLASSES ---

```
types.Operator):  
X mirror to the selected  
object.mirror_mirror_x"  
mirror X"
```

```
context):  
context.active_object is not
```

Program specification and System Diagram

1. The child switches on the system using the power switch. This provides power to the Arduino and all connected components including the OLED screen, RFID sensor, buttons, speaker, and solenoids.
2. The system waits for an RFID card to be scanned. If an authorised card is detected, the Arduino allows the game to start, plays a start-up sound, and shows a welcome message on the OLED screen. If an unauthorised card is scanned, the system plays an error sound and displays “Invalid Card – Try Again”. The game will not start until a valid card is scanned.
3. The OLED then asks a question in the format “How many [fruit]?”. The fruits and their correct numbers are stored in a predefined 2D list inside the Arduino program, for example apples = 1, bananas = 5, grapes = 10.
4. The child presses the counter button to increase the number shown on the OLED screen. If they make a mistake, the reset button returns the counter to zero. When they are confident, the child presses the submit button to confirm their answer.
5. The Arduino compares the child’s input with the stored correct number. If the answer is correct, a positive sound plays, the OLED shows “Correct!”, and the game moves on to the next fruit. If the answer is incorrect, a warning sound plays, the OLED shows “Try Again”, and the counter resets to zero so the child can retry.
6. The process repeats until all ten fruit questions have been answered correctly. When the child completes the game, the Arduino plays a victory sound, displays “Well Done! You Win!” on the OLED screen and activates the solenoids to unlock the till cash drawer.
7. After unlocking, the solenoids turn off and the drawer locks again. The system resets and requires the RFID card to be scanned again before another round can start.



Code Development - OLED

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
```

This imports the libraries needed for the program to communicate with the hardware

```
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1 // Reset pin not used
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
```

This sets up the OLED with its specification

```
void setup() {
  Serial.begin(9600);

  // Initialize display
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) { // 0x3C = I2C address
    Serial.println(F("SSD1306 allocation failed"));
    for (;;) // Don't proceed, loop forever
  }
}
```

This initialises the OLED and attempts to communicate with at screen grabbed its physical "hardware address" at 0x3D

```
display.clearDisplay();
display.setTextSize(2); // Text size
display.setTextColor(SSD1306_WHITE); // Color
display.setCursor(0, 10); // Position
display.println(F("Hello!"));
display.display();
}
```

This communicates with the OLED and tells it what to do (text size) and display ("Hello")

```
void loop() {
}
```


Code Development -MFRC522 (RFID) + LCD

```
#include <SPI.h>
#include <MFRC522.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#undef WHITE
#include "Waveshare_LCD1602_RGB.h"

Adafruit_SSD1306 oled(128,64,&Wire,-1);
MFRC522 rfid(53,49);
Waveshare_LCD1602_RGB lcd(16,2);
int r,g,b,t=0;

void setup(){
  Serial.begin(9600);
  SPI.begin();
  rfid.PCD_Init();
  oled.begin(SSD1306_SWITCHCAPVCC,0x3C);
  oled.clearDisplay(); oled.setTextSize(1);
  oled.setTextColor(SSD1306_WHITE);
  oled.setCursor(0,0); oled.println("RFID Ready");
  oled.setCursor(0,20); oled.println("Waiting...");
  oled.display();
  lcd.init();
  lcd.setCursor(0,0); lcd.send_string("RFID UID Disp");
  lcd.setCursor(0,1); lcd.send_string("Ready");
}

void loop(){
  r=abs(sin(3.14*t/180))*255;
  g=abs(sin(3.14*(t+60)/180))*255;
  b=abs(sin(3.14*(t+120)/180))*255;
  lcd.setRGB(r,g,b);
  t=(t+3)%360;
}
```

This imports the libraries needed for the program to communicate with the hardware

This sets up the OLED, LCD and MFRC522 with its specification

This communicates with the modules and initialises it

This communicates with the LCD and initialises it for the backlight

```
if(!rfid.PICC_IsNewCardPresent()||!rfid.PICC_ReadCardSerial())
{delay(80);return;}

oled.clearDisplay(); oled.setCursor(0,0);
oled.println("Reading..."); oled.display();

String u="";
for(byte i=0;i<rfid.uid.size;i++){
  u+=(rfid.uid.uidByte[i]<0x10?"0":"");
  u+=String(rfid.uid.uidByte[i],HEX);
}
u.toUpperCase();

oled.clearDisplay();
oled.setCursor(0,0); oled.println("Card detected!");
oled.setTextSize(2); oled.setCursor(0,20); oled.println("OK");
oled.setTextSize(1);
oled.display();

lcd.setCursor(0,0); lcd.send_string("UID:   ");
lcd.setCursor(0,1); lcd.send_string(u.c_str());

delay(1500);

oled.clearDisplay();
oled.setCursor(0,0); oled.println("RFID Ready");
oled.setCursor(0,20); oled.println("Waiting...");
oled.display();
lcd.setCursor(0,0); lcd.send_string("RFID UID Disp");
lcd.setCursor(0,1); lcd.send_string("Ready   ");
}
```

This checks for a RFID tag and tells the OLED to display "Reading"

This stores the Dips UID and tells the OLED to display the status "card detected"

This tells the LCD to display the UID

This resets the detecting

Code Development - Solenoids

```
#include <SPI.h>
#include <MFRC522.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#undef WHITE
#include "Waveshare_LCD1602_RGB.h"
#include <SoftwareSerial.h>
```

This imports the libraries needed for the program to communicate with the hardware

```
Adafruit_SSD1306 oled(128,64,&Wire,-1);
MFRC522 rfid(53,49);
Waveshare_LCD1602_RGB lcd(16,2);
const int solenoidPin = 7;
int r,g,b,t=0;
```

This sets up the OLED, LCD, MFRC522 and Solenoid with its specification

```
void setup() {
  Serial.begin(9600);
  SPI.begin();
  rfid.PCD_Init();
  oled.begin(SSD1306_SWITCHCAPVCC,0x3C);
  oled.clearDisplay();
  oled.setTextSize(1);
  oled.setTextColor(SSD1306_WHITE);
  oled.setCursor(0,0); oled.println("RFID Ready");
  oled.setCursor(0,20); oled.println("Waiting...");
  oled.display();
  lcd.init();
  lcd.setCursor(0,0); lcd.send_string("RFID UID Disp");
  lcd.setCursor(0,1); lcd.send_string("Ready");
}
```

This communicates with the modules and initialises it

```
void loop() {
  r = abs(sin(3.14*t/180))*255;
  g = abs(sin(3.14*(t+60)/180))*255;
  b = abs(sin(3.14*(t+120)/180))*255;
  lcd.setRGB(r,g,b);
  t = (t+3)%360;
}
```

This communicates with the LCD and initialises it for the backlight

```
if(!rfid.PICC_IsNewCardPresent() ||
!rfid.PICC_ReadCardSerial()){
  delay(80);
  return;
}
```

This checks for a RFID tag and tells the OLED to display "Reading"

```
oled.clearDisplay();
oled.setCursor(0,0);
oled.println("Reading...");
oled.display();
```

```
String u = "";
for(byte i=0; i < rfid.uid.size; i++){
  u += (rfid.uid.uidByte[i] < 0x10 ? "0" : "");
  u += String(rfid.uid.uidByte[i], HEX);
}
u.toUpperCase();
```

This stores the UID and tells the OLED to display the status "card detected"

```
oled.clearDisplay();
oled.setCursor(0,0); oled.println("Card detected!");
oled.setTextSize(2);
oled.setCursor(0,20); oled.println("OK");
oled.setTextSize(1);
oled.display();
```

```
lcd.setCursor(0,0); lcd.send_string("UID:   ");
lcd.setCursor(0,1); lcd.send_string(u.c_str());

digitalWrite(solenoidPin, HIGH);
delay(3000);
digitalWrite(solenoidPin, LOW);
delay(3000)

oled.clearDisplay();
oled.setCursor(0,0); oled.println("RFID Ready");
oled.setCursor(0,20); oled.println("Waiting...");
oled.display();

lcd.setCursor(0,0); lcd.send_string("RFID UID Disp");
lcd.setCursor(0,1); lcd.send_string("Ready   ");
}
```

This tells the LCD to display the UID

This makes the Solenoid to unlock for 3 seconds

This resets the detecting



Main Game Logic

Coded in Python-Style Code

This stores all the data for my variables in a 2d List

This stores all the counter values when the buttons have been pressed

It checks if the counter button has been pressed and updates the counter

It checks if the answer is right, it plays a "correct tone"

It checks if the answer is wrong, it plays a "wrong tone"

After 10 correct answers in a row, it plays a "well done tune" and unlocks the solenoid

```
fruits = [
    ["Watermelons", 1], ["Pineapples", 2], ["Mangos", 3], ["Apples", 4],
    ["Bananas", 5], ["Oranges", 6], ["Kiwis", 7], ["Plums", 8],
    ["Strawberries", 9], ["Grapes", 10]
]

consecutive_correct = 0
counter_value = 0
MAX_CORRECT = 10

current_fruit = RANDOM_CHOICE_FROM(fruits)
oled.display("How many " + current_fruit[0] + " are there?")
lcd.display("Progress: [" + "#" * consecutive_correct + "-" * (MAX_CORRECT - consecutive_correct) + "]" + "Count: " + str(counter_value))

while consecutive_correct < MAX_CORRECT:

    if counter_button_pressed:
        counter_value = counter_value + 1
        lcd.display("Progress: [" + "#" * consecutive_correct + "-" * (MAX_CORRECT - consecutive_correct) + "]" + "Count: " + str(counter_value))

    if next_button_pressed:
        correct_number = current_fruit[1]

        if counter_value == correct_number:
            dfmini.play(right_track)
            consecutive_correct = consecutive_correct + 1

        else:
            dfmini.play(wrong_track)
            consecutive_correct = 0

        counter_value = 0
        current_fruit = RANDOM_CHOICE_FROM(fruits)
        oled.display("How many " + current_fruit[0] + " are there?")
        lcd.display("Progress: [" + "#" * consecutive_correct + "-" * (MAX_CORRECT - consecutive_correct) + "]" + "Count: " + str(counter_value))

    dfmini.play(well_done_track)
    solenoid.unlock()
```

It asks the question

It checks if the next button has been pressed and checks if the answer is correct

It asks the next question

How it relates to the final code?

In the previous slide, core logic of the game is written in Python style code. This interacts with code that controls operation of solenoid, DfPlayerMini, LCD and buttons, which is given at Slide # 43 onwards in the final working code.

Why I chose Python-style code?

I chose to use Python for this stage because its straightforward syntax allowed me to focus entirely on the game's "flow" and behaviour rather than complex hardware integration. Since I was already familiar with this language, it was the most efficient tool for agile testing and allowed me to virtually visualize the game running in a text-based format.

Coding tutorials and Data Sheets

Tutorials :

Arduino - <https://lastminuteengineers.com/arduino-mega-pinout/>

LCD - <https://lastminuteengineers.com/i2c-lcd-arduino-tutorial/>

RFID - <https://lastminuteengineers.com/how-rfid-works-rc522-arduino-tutorial/>

DFPlayerMini - <https://www.instructables.com/MP3-Player-With-Arduino-Using-DF-Player-Mini/>

Solenoids - <https://www.instructables.com/Controlling-a-solenoid-valve-with-an-Arduino/>

Datasheets :

Arduino - <https://docs.arduino.cc/resources/datasheets/A000067-datasheet.pdf>

Waveshare LCD - https://files.waveshare.com/upload/2/2e/LCD1602_RGB_Module.pdf

MFRC522 - <https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf>

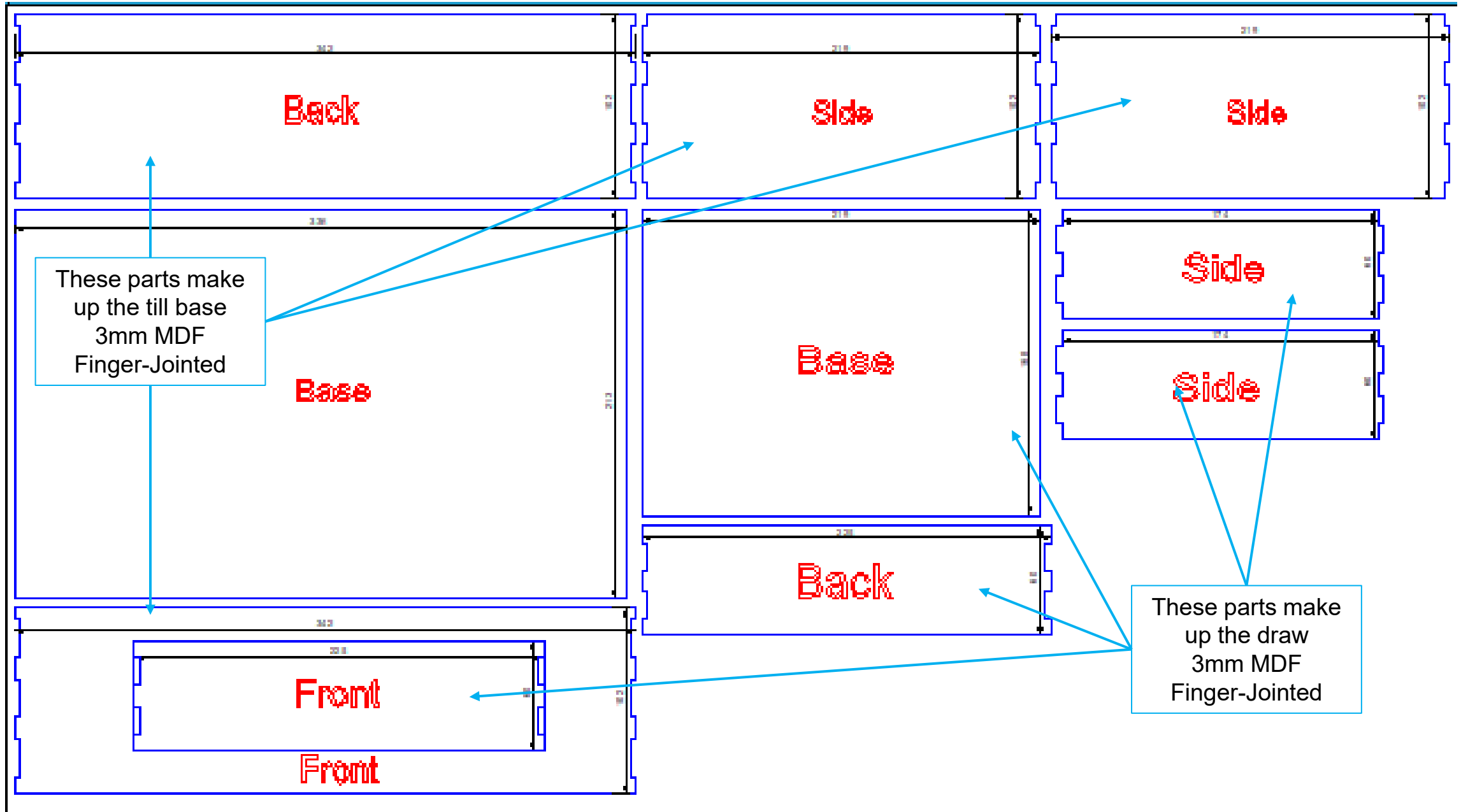
DFPlayerMini - <https://picaxe.com/docs/spe033.pdf>

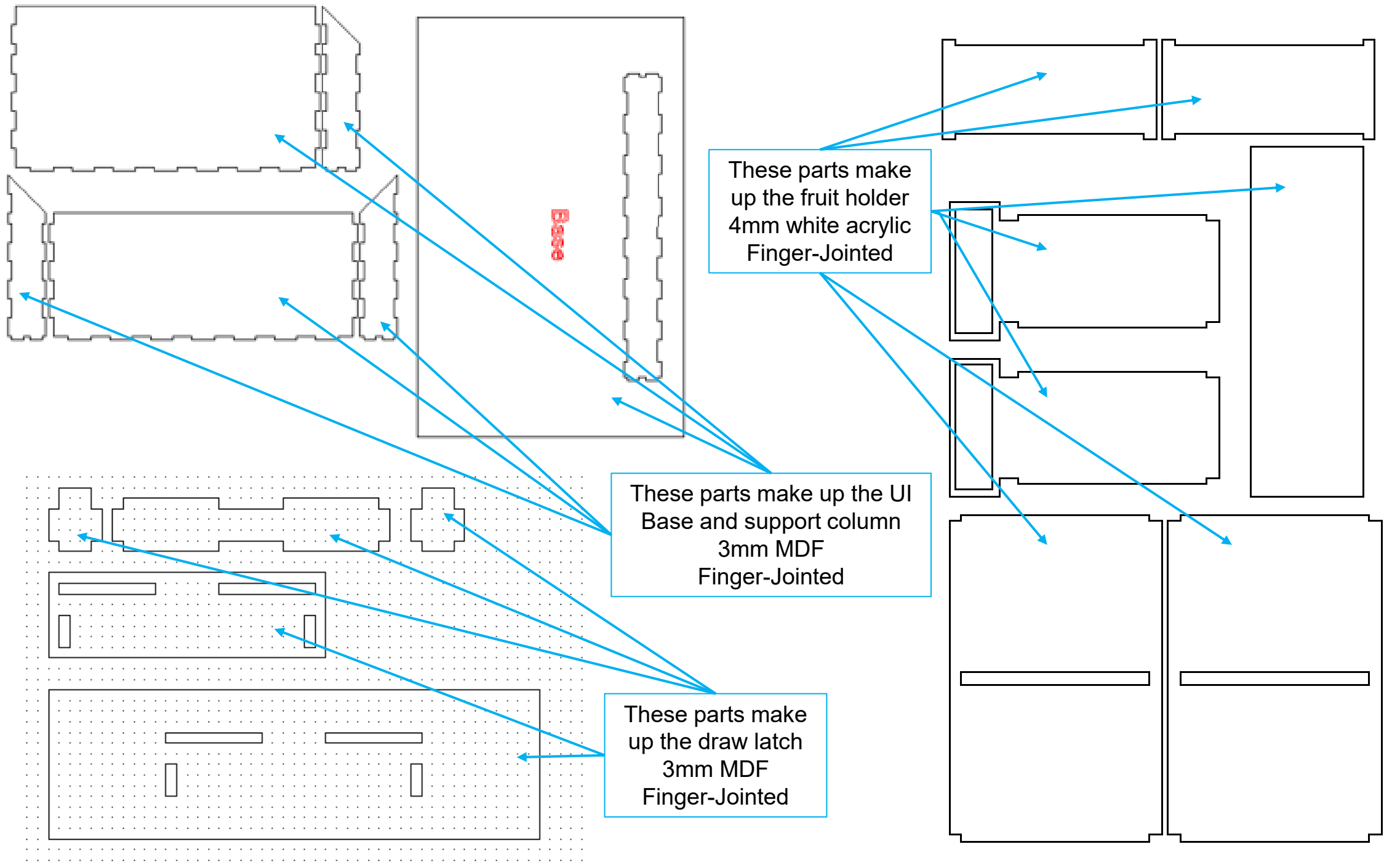
Solenoids - https://cdn.sparkfun.com/assets/9/b/9/e/0/NI-11_instruction.pdf

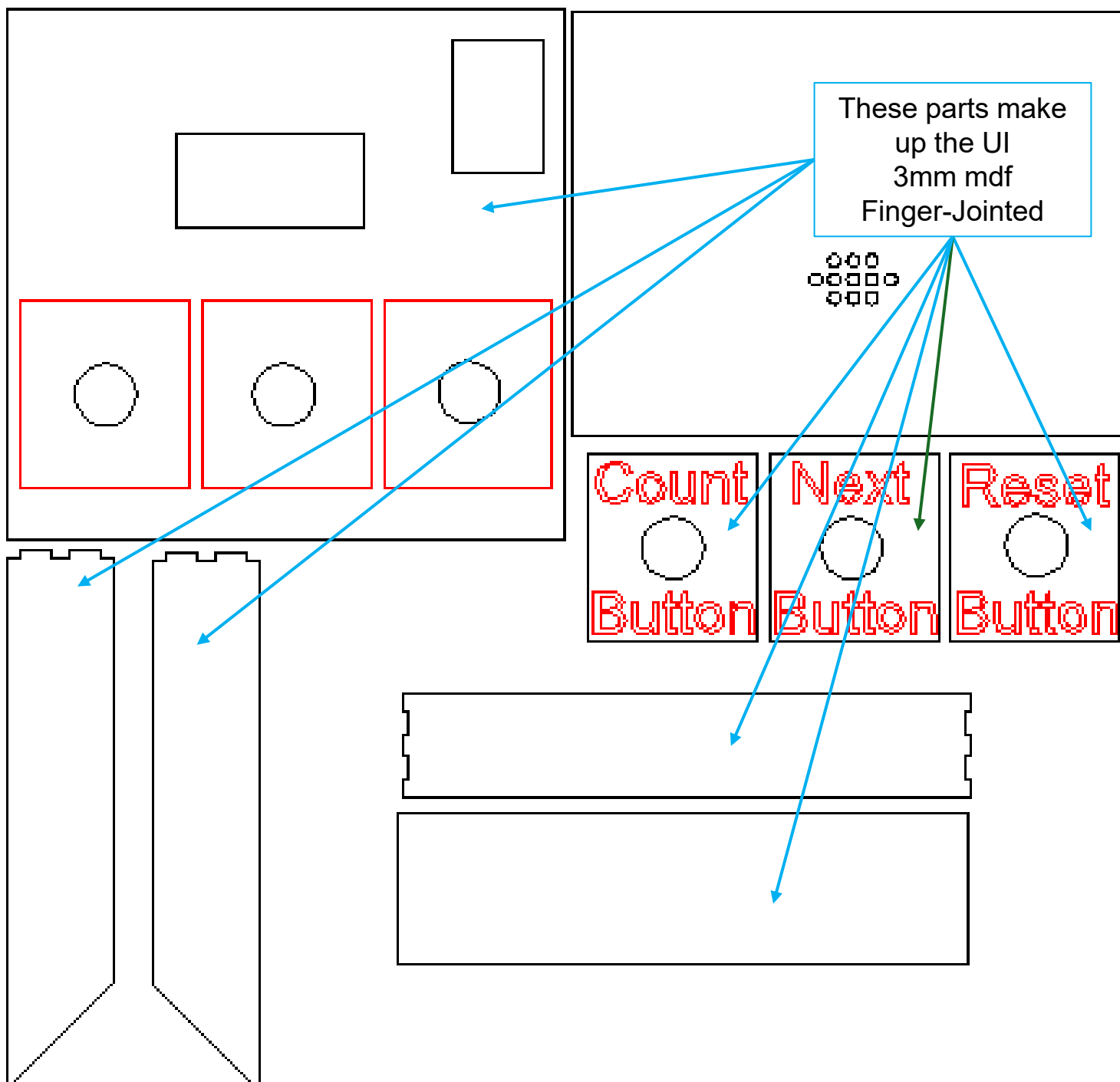
1N007 Diode - <https://global.oup.com/us/companion.websites/fdscontent/uscompanion/us/pdf/microcircuits/students/diode/1n4004-general.pdf>

IRL44N MOSFET - <https://www.vishay.com/docs/91328/irlz44.pdf>

LASER CUT PARTS







This make the
black letters
3mm black acrylic

Count Next Reset
Button Button Button

Ohm's Law can be used to determine the current flowing through the solenoid.

Formula: $V = I \times R$

Rearranged: $I = V / R$

- Supply voltage = 12V
- Solenoid resistance = 24Ω

Substituting values:

$$I = 12 / 24$$

$$I = 0.5A$$

So, the solenoid draws 0.5A (500mA) when activated.

From the datasheet online, the IRLZ44N can handle:

- Maximum drain current (I_d) ≈ 47A
- Logic-level gate (fully turns on at 5V)

Since:

Solenoid current = 0.5A

MOSFET maximum = 47A

0.5A is well within safe operating limits.

Therefore, the MOSFET is suitable.

Power dissipated in the MOSFET

$$P = I^2 \times R_{ds(on)}$$

The IRLZ44N has $R_{ds(on)} \approx 0.022\Omega$

$$P = (0.5)^2 \times 0.022$$

$$P = 0.25 \times 0.022$$

$$P = 0.0055W$$

So, only 0.0055W of heat is generated.

This is very small, so no heatsink is required.

Calculations

determine the requirements of the Solenoid and Flyback Diode

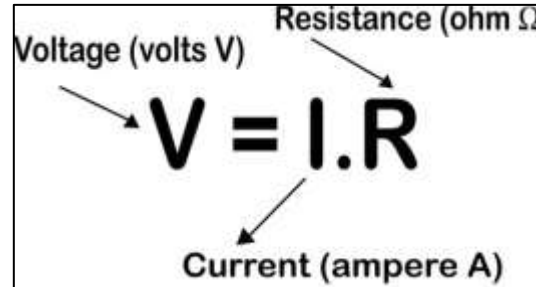


Diagram illustrating the formula $V = I \cdot R$ with labels:

- Voltage (volts V)
- Resistance (ohm Ω)
- Current (ampere A)

Current Check in Diode

From my previous calculation:

Solenoid current = 0.5A

The 1N4007 diode has:

- Maximum forward current = 1A
- Peak reverse voltage = 1000V

Since:

$$0.5A < 1A$$

The diode can safely handle the current.

Reverse Voltage Check

My supply voltage is 12V.

The 1N4007 can withstand up to 1000V reverse voltage.

Since:

$$12V \ll 1000V$$

The diode is more than suitable for this circuit.

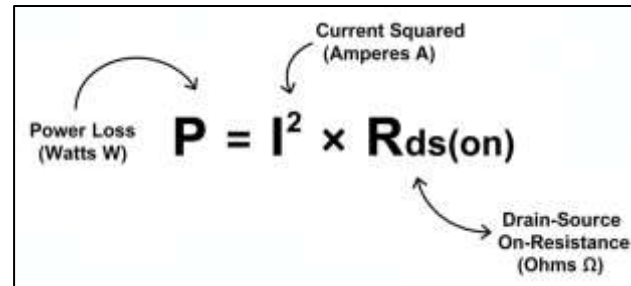


Diagram illustrating the formula $P = I^2 \times R_{ds(on)}$ with labels:

- Power Loss (Watts W)
- Current Squared (Amperes A)
- Drain-Source On-Resistance (Ohms Ω)

Power Dissipation in the Diode

$$Power = V \times I$$

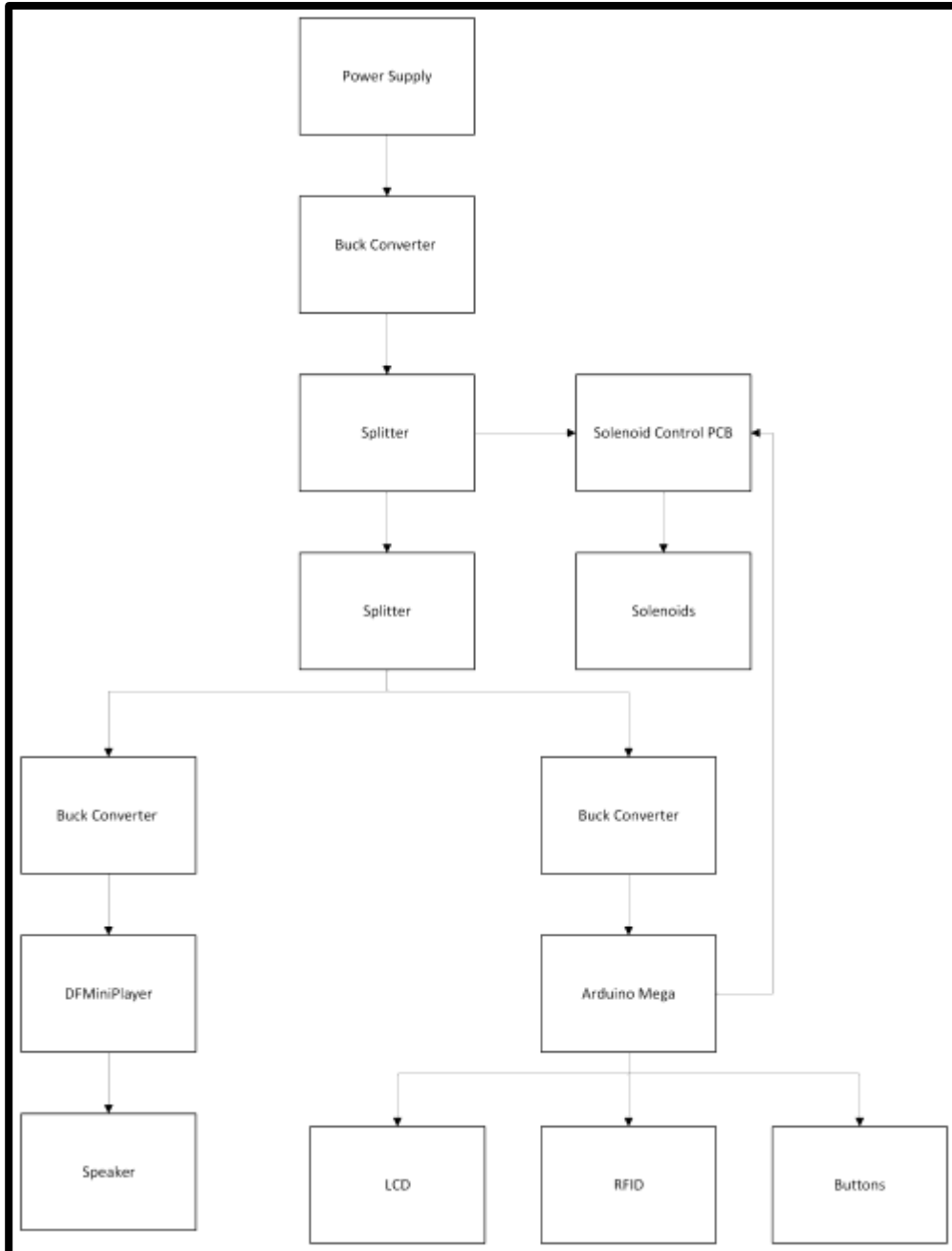
The forward voltage drop of a 1N4007 is approximately 0.7V.

$$P = 0.7 \times 0.5$$

$$P = 0.35W$$

The diode can safely dissipate this power in short pulses when the solenoid turns on. So no heatsink is required

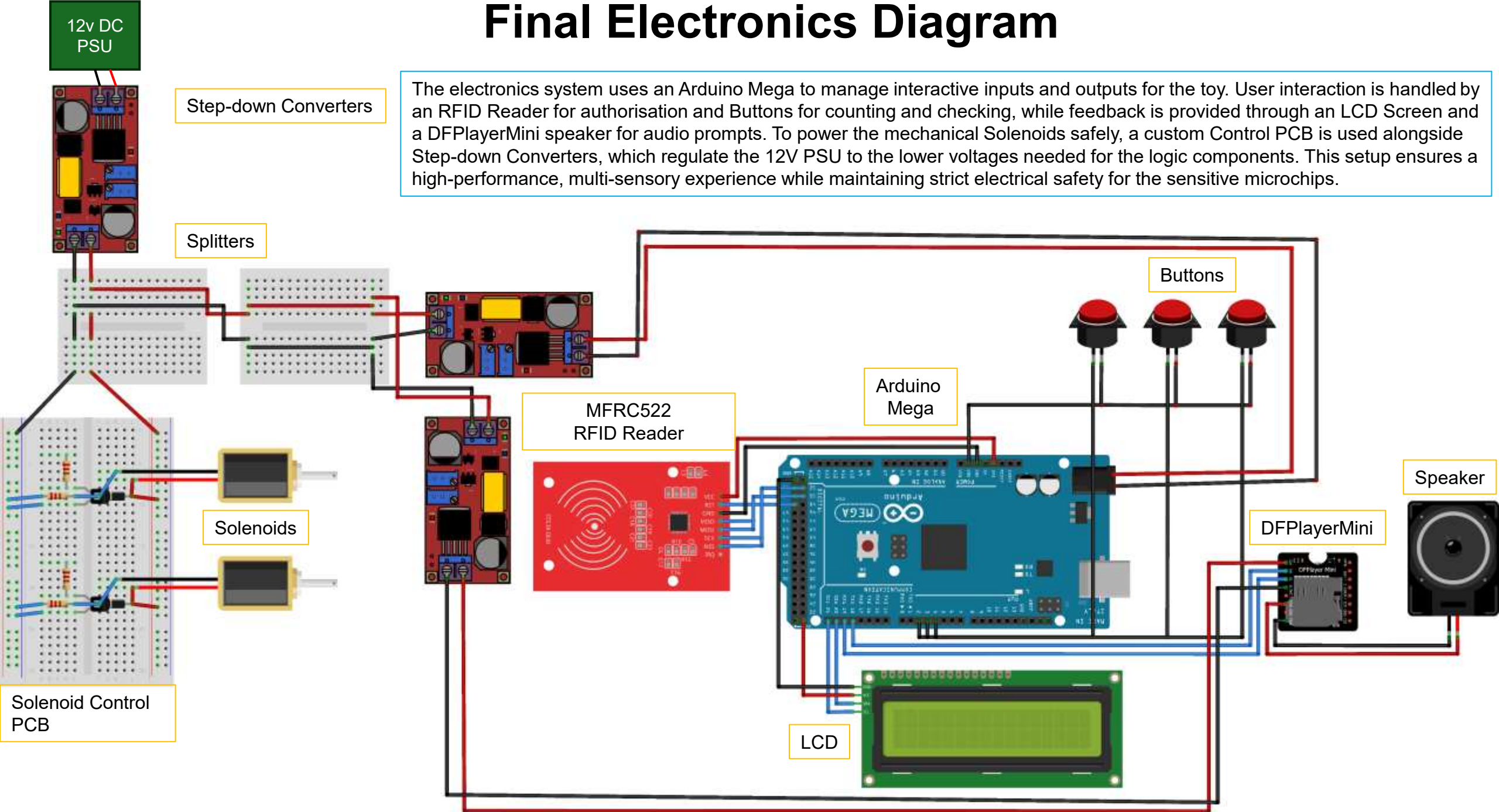
Electronic Block System Diagram



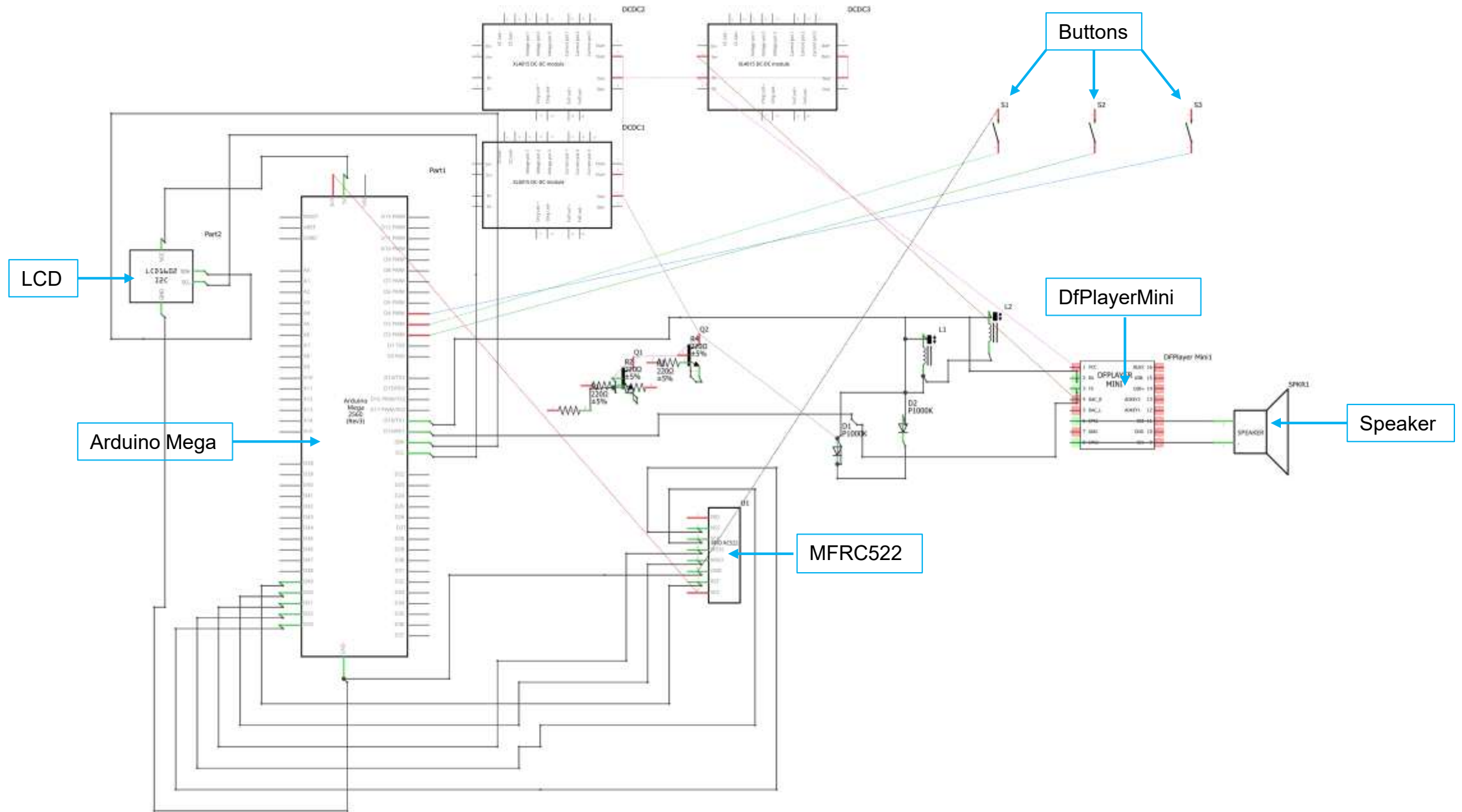
1. The power supply is plugged in.
2. The system is powered on.
3. The electricity flows through the first buck converter to set the voltage at 12v and the current at 4 amps
4. Then it goes through a splitter where a single positive and negative wire is split into 2 positive and 2 negative wires.
5. One set of these positive and negative wires go to the solenoid control PCB
6. While the other set goes to another splitter.
7. One set from the 2nd splitter goes to the Arduino via a buck converter which provide a steady supply of 9v and 1 amp to the Arduino Mega
8. The other set from the 2nd splitter goes to the DFMiniPlayer. Since the DFMiniPlayer sensitive to voltage drops and noisy current I have use another buck converter to stabilize the flow of electricity.

1. Once the entire system is powered, the Arduino starts the game.
2. First the RFID is active until the correct ID card has been scanned
3. Then the game starts, everything is displayed on the LCD
4. The child will play the game using the buttons
5. The DFPlayerMini is used to carry out various tasks, such as asking the questions, playing the correct tone and the congratulatory message when the child wins the game. The sound is produced via the speaker
6. If they get win the game, the Mega will send a signal to the solenoid control board, through which the MOSFETs power the solenoids

Final Electronics Diagram



Final Electronics Schematic





Final Working Code

```

1  #include <SPI.h>
2  #include <MFRC522.h>
3  #include <Wire.h>
4  #include "Waveshare_LCD1602_RGB.h"
5  #include <DFRobotDFPlayerMini.h>
6
7  // ===== DFMini MP3 Player ===== //
8  DFRobotDFPlayerMini mp3;
9
10 // ===== RFID ===== //
11 #define RFID_SS 53
12 #define RFID_RST 49
13 MFRC522 rfid(RFID_SS, RFID_RST);
14
15
16 byte allowedUID[] = {0x85, 0x7A, 0xFE, 0x29};
17
18 // ===== LCD ===== //
19 Waveshare_LCD1602_RGB lcd(16, 2);
20
21 // ===== BUTTONS ===== //
22 #define BTN_COUNT 2
23 #define BTN_NEXT 3
24 #define BTN_RESET 4
25
26 bool lastCountState = HIGH;
27 bool lastNextState = HIGH;
28 bool lastResetState = HIGH;
29
30 // ===== SOLENOIDS ===== //
31 #define SOL1 9
32 #define SOL2 8
33
34 // ===== GAME ===== //
35 enum State { WAIT_CARD, GAME, WIN };
36 State state = WAIT_CARD;
37 unsigned long now;

```

This bit tells the program to download and include these drivers

This tells the program what components and what pins they are using

```

38 unsigned long btnTimer = 0;
39 unsigned long audioTimer = 0;
40 unsigned long debounceDelay = 200;
41 bool isPlayingAudio = false;
42 int counter = 0;
43 int qIndex = 0;
44 int currentTrack = 0;
45 bool lcdWorking = false;
46
47 // Denial display
48 bool showingDeny = false;
49 unsigned long denyTimer = 0;
50
51 int totalQuestions = 10;
52
53 // Win state timers
54 unsigned long winStartTime = 0;
55 const unsigned long winTotalDuration = 6000;
56 const unsigned long solenoidDuration = 4000;
57
58
59 unsigned long lastWelcomePlayTime = 0;
60 const unsigned long welcomeCooldown = 5000;
61 const unsigned long welcomeInterval = 10000;
62
63 // ===== FRUITS ===== //
64 struct Fruit {
65     const char* name;
66     int qty;
67     int audioFile; // Audio file number
68 };
69
70
71 Fruit fruits[] = {
72     {"Apples", 1, 5}, // File 0005.mp3 = 1 apple
73     {"Pineapples", 2, 6}, // File 0006.mp3 = 2 pineapples
74     {"Oranges", 3, 7}, // File 0007.mp3 = 3 oranges

```

This bit sets up all the constants and the variables in the program


```

75 {"Mango pieces", 4, 8}, // File 0008.mp3 = 4 mango pieces
76 {"Bananas", 5, 9}, // File 0009.mp3 = 5 bananas
77 {"Watermelon slices", 6, 10}, // File 0010.mp3 = 6 watermelon slices
78 {"Kiwi slices", 7, 11}, // File 0011.mp3 = 7 kiwi slices
79 {"Plums", 8, 12}, // File 0012.mp3 = 8 plums
80 {"Strawberries", 9, 13}, // File 0013.mp3 = 9 strawberries
81 {"Grapes", 10, 14} // file 0014.mp3 = 10 grapes
82 };
83
84 int order[10];
85
86 // ===== AUDIO ===== //
87 unsigned long lastPlayTime = 0;
88
89 void playAudio(int track) {
90
91     if (track == currentTrack && (now - lastPlayTime) < 500) {
92         Serial.println("[AUDIO] Skipped duplicate request");
93         return;
94     }
95     lastPlayTime = now;
96
97     if (track >= 1 && track <= 16) {
98         Serial.print("[AUDIO] Playing: ");
99         if(track < 10) Serial.print("0");
100         if(track < 100) Serial.print("0");
101         Serial.print(track);
102         Serial.print(".mp3");
103
104
105         mp3.stop();
106         delay(50);
107
108
109         mp3.play(track);
110         isPlayingAudio = true;
111         audioTimer = now;

```

This sets up the fruit variables

This sections defines the audio functions that the program can reuse multiple times

```

112     currentTrack = track;
113
114
115     switch(track) {
116         case 1: Serial.println(" (tap the card)"); break;
117         case 2: Serial.println(" (access granted)"); break;
118         case 3: Serial.println(" (access denied)"); break;
119         case 4: Serial.println(" (well done)"); break;
120         case 5: Serial.println(" (apples - answer: 1)"); break;
121         case 6: Serial.println(" (pineapples - answer: 2)"); break;
122         case 7: Serial.println(" (oranges - answer: 3)"); break;
123         case 8: Serial.println(" (mango - answer: 4)"); break;
124         case 9: Serial.println(" (bananas - answer: 5)"); break;
125         case 10: Serial.println(" (watermelon - answer: 6)"); break;
126         case 11: Serial.println(" (kiwi - answer: 7)"); break;
127         case 12: Serial.println(" (plum - answer: 8)"); break;
128         case 13: Serial.println(" (strawberries - answer: 9)"); break;
129         case 14: Serial.println(" (grapes - answer: 10)"); break;
130         case 15: Serial.println(" (correct tone)"); break;
131         case 16: Serial.println(" (incorrect tone)"); break;
132         default: Serial.println(); break;
133     }
134 }
135
136
137 void stopAudio() {
138     mp3.stop();
139     isPlayingAudio = false;
140     Serial.println("[AUDIO] Stopped");
141 }
142
143 // audio completion check //
144 bool checkAudioFinished() {
145     if (!isPlayingAudio) return true;
146
147     unsigned long estimatedDuration;

```

This outputs what track is being played to the serial monitor

```

150     if (currentTrack == 1) estimatedDuration = 3000;
151     else if (currentTrack == 2) estimatedDuration = 2000;
152     else if (currentTrack == 3) estimatedDuration = 2000;
153     else if (currentTrack == 4) estimatedDuration = 5000;
154     else if (currentTrack >= 5 && currentTrack <= 14) estimatedDuration = 4000;
155     else if (currentTrack == 15 || currentTrack == 16) estimatedDuration = 1000;
156     else estimatedDuration = 4000;
157
158
159     if (now - audioTimer > estimatedDuration + 500) {
160         isPlayingAudio = false;
161         return true;
162     }
163
164     return false;
165 }
166
167 void playWelcomeMessage() {
168
169     if (!isPlayingAudio && (now - lastWelcomePlayTime > welcomeCooldown)) {
170         lastWelcomePlayTime = now;
171         playAudio(1); // "tap the card"
172     }
173 }
174
175 void playAccessGranted() {
176     playAudio(2); // "access granted"
177 }
178
179 void playAccessDenied() {
180     playAudio(3); // "access denied"
181 }
182
183 void playWinMessage() {
184     playAudio(4); // "well done"
185 }

```

```

187 void playCorrectTone() {
188     playAudio(15); // "correct tone"
189 }
190
191 void playIncorrectTone() {
192     playAudio(16); // "incorrect tone"
193 }
194
195 void playQuestionAudio() {
196     int idx = order[qIndex];
197     int track = fruits[idx].audioFile;
198     playAudio(track);
199 }
200
201 // ===== LCD =====
202 void showScanMessage() {
203     if(!lcdWorking) return;
204     lcd.setRGB(0, 0, 255);
205     lcdClear();
206     lcd.setCursor(0, 0);
207     lcd.send_string("Scan a card");
208 }
209
210 void showAccessGrantedScreen() {
211     if(!lcdWorking) return;
212     lcd.setRGB(0, 255, 0);
213     lcdClear();
214     lcd.setCursor(0, 0);
215     lcd.send_string("Access Granted!");
216     lcd.setCursor(0, 1);
217     lcd.send_string("Good job");
218     delay(2000);
219 }
220
221 void showDenyMessage() {
222     if(!lcdWorking) return;

```

This sections defines the LCD functions that the program can reuse multiple times


```

222     if(!lcdWorking) return;
223     lcd.setRGB(255, 0, 0);
224     lcdClear();
225     lcd.setCursor(0, 0);
226     lcd.send_string("Access Denied");
227     lcd.setCursor(0, 1);
228     lcd.send_string("Wrong Card");
229 }

```

```

231 void shuffleQ() {
232     randomSeed(analogRead(0));
233     for(int i = 0; i < 10; i++) order[i] = i;
234     for(int i = 9; i > 0; i--) {
235         int j = random(i + 1);
236         int t = order[i];
237         order[i] = order[j];
238         order[j] = t;
239     }
240 }

```

This bit shuffles the question that will be asked

```

242 void lcdClear() {
243     if(!lcdWorking) return;
244     lcd.setCursor(0, 0);
245     lcd.send_string(" ");
246     lcd.setCursor(0, 1);
247     lcd.send_string(" ");
248 }

```

```

250 void lcdCounter() {
251     if(!lcdWorking) return;
252     char buffer[17];
253     snprintf(buffer, sizeof(buffer), "Count: %2d", counter);
254     lcd.setCursor(0, 0);
255     lcd.send_string(buffer);
256 }

```

```

257 void showQuestion() {
258

```

```

259     int idx = order[qIndex];
260
261     // Display on LCD
262     if(lcdWorking) {
263         lcdClear();
264         lcd.setCursor(0, 0);
265         lcd.send_string("How many?");
266         lcd.setCursor(0, 1);
267         lcd.send_string(fruits[idx].name);
268     }

```

```

271     Serial.println("");
272     Serial.print("=== Question ");
273     Serial.print(qIndex + 1);
274     Serial.print("/");
275     Serial.print(totalQuestions);
276     Serial.print(" ===");
277

```

```

278     int audioFile = fruits[idx].audioFile;
279     Serial.print(" File ");
280     if(audioFile < 10) Serial.print("0");
281     Serial.print(audioFile);
282     Serial.print(".mp3: How many ");
283     Serial.print(fruits[idx].name);
284     Serial.print("? (Correct: ");
285     Serial.print(fruits[idx].qty);
286     Serial.println(")");
287

```

This bit prints the question and the answer to the serial

```

288     /
289     delay(300);
290     playQuestionAudio();
291 }

```

```

293 // ===== LOADING SCREEN ===== //

```

```

294 void showLoadingScreen() {
295     if(!lcdWorking) return;

```

```

296   lcd.setRGB(0, 0, 255); [
297   lcd.setCursor(0, 0);
298   lcd.send_string("Loading...");
299
300
301   lcd.setCursor(0, 1);
302   lcd.send_string("[");
303   for (int i = 1; i <= 14; i++) {
304       lcd.setCursor(i, 1);
305       lcd.send_string(" ");
306   }
307   lcd.setCursor(15, 1);
308   lcd.send_string("]");
309
310
311   for (int i = 1; i <= 14; i++) {
312       lcd.setCursor(i, 1);
313       lcd.send_string("#");
314       delay(200);
315   }
316   lcdClear();

```

This chunk of code handles the dynamic loading screen

```

317
318
319 // ===== DIFFICULTY SELECTION ===== //
320 void selectDifficultyAfterCard() {
321     if(!lcdWorking) return;
322     lcd.setRGB(255, 255, 0); // Yellow
323     lcd.setCursor(0, 0);
324     lcd.send_string("Easy Med Hard");
325     lcd.setCursor(0, 1);
326     lcd.send_string("Count Next Reset");
327     Serial.println("Choose difficulty: Press Count (Easy), Next (Medium), Reset (Hard)");
328
329     while (true) {
330         if (digitalRead(BTN_COUNT) == LOW) {
331             delay(50);
332             if (digitalRead(BTN_COUNT) == LOW) {

```

This section allows the user to pick a difficulty

```

333         while (digitalRead(BTN_COUNT) == LOW) delay(10);
334         totalQuestions = 3;
335         Serial.println("Easy mode selected: 3 questions");
336         break;
337     }
338 }
339 if (digitalRead(BTN_NEXT) == LOW) {
340     delay(50);
341     if (digitalRead(BTN_NEXT) == LOW) {
342         while (digitalRead(BTN_NEXT) == LOW) delay(10);
343         totalQuestions = 7;
344         Serial.println("Medium mode selected: 7 questions");
345         break;
346     }
347 }
348 if (digitalRead(BTN_RESET) == LOW) {
349     delay(50);
350     if (digitalRead(BTN_RESET) == LOW) {
351         while (digitalRead(BTN_RESET) == LOW) delay(10);
352         totalQuestions = 10;
353         Serial.println("Hard mode selected: 10 questions");
354         break;
355     }
356 }
357 delay(50);
358 }
359
360
361 lcdClear();
362 lcdCounter();
363 }
364
365 // ===== GAME WON EFFECT ===== //
366 void setRainbowColor(unsigned long elapsed, unsigned long totalDuration) {
367
368     float hue = (float)elapsed / totalDuration * 360.0;
369

```



```

370 int r, g, b;
371 int segment = (int)hue / 60;
372 float frac = (hue / 60.0) - segment;
373
374 switch(segment % 6) {
375     case 0: r = 255; g = (int)(255 * frac); b = 0; break;
376     case 1: r = (int)(255 * (1 - frac)); g = 255; b = 0; break;
377     case 2: r = 0; g = 255; b = (int)(255 * frac); break;
378     case 3: r = 0; g = (int)(255 * (1 - frac)); b = 255; break;
379     case 4: r = (int)(255 * frac); g = 0; b = 255; break;
380     default: r = 255; g = 0; b = (int)(255 * (1 - frac)); break;
381 }
382
383 lcd.setRGB(r, g, b);
384 }
385
386
387
388
389 // ===== SETUP =====
390 void setup() {
391     Serial.begin(9600);
392     delay(1000);
393     Serial.println("=====");
394     Serial.println("FRUIT COUNTING GAME");
395     Serial.println("=====");
396
397     randomSeed(analogRead(0));
398
399     // Buttons
400     pinMode(BTN_COUNT, INPUT_PULLUP);
401     pinMode(BTN_NEXT, INPUT_PULLUP);
402     pinMode(BTN_RESET, INPUT_PULLUP);
403     Serial.println("Buttons ready");
404
405     // Solenoids
406     pinMode(SOL1, OUTPUT);

```

This bit handles the rainbow colourful animation when the game is won

This next section sets up the game by initializing the modules.

```

407 pinMode(SOL2, OUTPUT);
408 digitalWrite(SOL1, LOW);
409 digitalWrite(SOL2, LOW);
410 Serial.println("Solenoids ready");
411
412 // I2C and LCD
413 Serial.println("Initializing LCD...");
414 Wire.begin();
415 Wire.setClock(100000L);
416 delay(200);
417
418 lcd.init();
419 delay(300);
420 lcdWorking = true;
421
422
423 showLoadingScreen();
424
425 // RFID
426 Serial.println("Initializing RFID...");
427 SPI.begin();
428 delay(50);
429 rfid.PCD_Init();
430 delay(100);
431
432 byte version = rfid.PCD_ReadRegister(rfid.VersionReg);
433 if (version == 0x00 || version == 0xFF) {
434     Serial.println("WARNING: RFID communication failed");
435 } else {
436     Serial.print("RFID ready - Version: 0x");
437     Serial.println(version, HEX);
438 }
439
440 // DFMini MP3 Player
441 Serial.println("Initializing DFMini MP3...");
442 Serial1.begin(9600);
443 delay(300);

```



```

444     if (!mp3.begin(Serial1)) {
445         Serial.println("ERROR: Unable to initialize DFMini player!");
446     } else {
447         Serial.println("DFMini MP3 player ready");
448         mp3.volume(30);
449         mp3.EQ(DFPLAYER_EQ_NORMAL);
450         mp3.outputDevice(DFPLAYER_DEVICE_SD);
451     }
452
453
454
455     shuffleQ();
456     state = WAIT_CARD;
457
458
459     showScanMessage();
460     playWelcomeMessage();
461
462     Serial.println("=====");
463     Serial.println("GAME READY - Tap card to start");
464     Serial.println("=====");
465
466
467
468
469
470 // ===== LOOP ===== //
471 void loop() {
472     now = millis();
473
474
475     if (isPlayingAudio && checkAudioFinished()) {
476         isPlayingAudio = false;
477     }
478
479
480     if (showingDeny && now >= denyTimer) {

```

```
481 showingDeny = false;
482 showScanMessage();
483
484
485
```

```
// ----- WAIT CARD ----- //
```

```
if(state == WAIT_CARD) {
```

```
    if(!showingDeny && now - lastWelcomePlayTime > welcomeInterval && !isPlayingAudio) {
        playWelcomeMessage();
    }
}
```

```
if (!showingDeny && rfid.PICC_IsNewCardPresent()) {
```

```
    if (rfid.PICC_ReadCardSerial()) {
```

```
        Serial.print("[RFID] CARD DETECTED: ");
```

```
        for(byte i = 0; i < rfid.uid.size; i++) {
```

```
            if(rfid.uid.uidByte[i] < 0x10) Serial.print("0");
```

```
            Serial.print(rfid.uid.uidByte[i], HEX);
```

```
            if(i < rfid.uid.size - 1) Serial.print(" ");
```

```
        }
```

```
        Serial.println();
```

```
        bool cardAllowed = true;
```

```
        if (rfid.uid.size != 4) {
```

```
            cardAllowed = false;
```

```
        } else {
```

```
            for (byte i = 0; i < 4; i++) {
```

```
                if (rfid.uid.uidByte[i] != allowedUID[i]) {
```

```
                    cardAllowed = false;
```

```
                    break;
```

```
                }
```

```
            }
```

```
        }
```

```
        rfid.PICC_HaltA();
```

```
        rfid.PCD_StopCrypto1();
```

This section waits till a card is tapped and validates it

```

518 if (cardAllowed) {
519     Serial.println("[RFID] ALLOWED CARD - Starting game");
520     playAccessGranted();
521
522     showAccessGrantedScreen();
523
524     delay(500);
525
526     selectDifficultyAfterCard();
527
528     state = GAME;
529     counter = 0;
530     qIndex = 0;
531
532     if(lcdWorking) {
533         lcd.setRGB(0, 0, 255);
534         lcdCounter();
535     }
536
537     Serial.println("=====");
538     Serial.println("[GAME STARTED]");
539     Serial.print("You need ");
540     Serial.print(totalQuestions);
541     Serial.println(" correct answers.");
542     Serial.println("=====");
543
544     delay(1000);
545     showQuestion();
546 } else {
547     Serial.println("[RFID] UNAUTHORIZED CARD - Access denied");
548     playAccessDenied();
549
550     showDenyMessage();
551     showingDeny = true;
552     denyTimer = now + 2000;
553 }
554

```

This bit registers what buttons are press and the actions they carry out

```

555 }
556
557
558
559
560
561 // ----- GAME -----
562 else if(state == GAME) {
563     if(now - btnTimer > debounceDelay) {
564         int countState = digitalRead(BTN_COUNT);
565         int nextState = digitalRead(BTN_NEXT);
566         int resetState = digitalRead(BTN_RESET);
567
568         // Count button
569         if (countState == LOW && lastCountState == HIGH) {
570             btnTimer = now;
571             counter++;
572             Serial.print("[COUNT] -> ");
573             Serial.println(counter);
574             lcdCounter();
575         }
576
577         // Reset button
578         else if (resetState == LOW && lastResetState == HIGH) {
579             btnTimer = now;
580             counter = 0;
581             Serial.println("[RESET] Counter = 0");
582             lcdCounter();
583         }
584
585         // Next button
586         else if (nextState == LOW && lastNextState == HIGH) {
587             btnTimer = now;
588             Serial.print("[ANSWER] ");
589             Serial.print(counter);
590
591             int correct = fruits[order[qIndex]].qty;
592             int audioFile = fruits[order[qIndex]].audioFile;
593
594             if(counter == correct) {

```

This is where the actually game starts


```

592 Serial.println(" - CORRECT! ✓");
593 Serial.print("File ");
594 if(audioFile < 10) Serial.print("0");
595 Serial.print(audioFile);
596 Serial.print(".mp3 = ");
597 Serial.print(correct);
598 Serial.print(" (");
599 Serial.print(audioFile);
600 Serial.print(" - 4)");
601
602 playCorrectTone();
603
604 qIndex++;
605 counter = 0;
606 lcdCounter();
607
608 if(qIndex >= totalQuestions) {
609     state = WIN;
610     winStartTime = now;
611
612     digitalWrite(SOL1, HIGH);
613     digitalWrite(SOL2, HIGH);
614
615     Serial.println("");
616     Serial.println("=====");
617     Serial.println("*** YOU WIN! ***");
618     Serial.println("CONGRATULATIONS!");
619     Serial.println("=====");
620
621     playWinMessage();
622
623     if(lcdWorking) {
624         lcdClear();
625         lcd.setCursor(0, 0);
626         lcd.send_string("YOU WIN!");

```

This checks if they have answered the right number of questions for their difficulty

This bit triggers all the winning actions such as the colourful LCD, Audio and solenoids

```

629         lcd.setCursor(0, 1);
630         lcd.send_string("CONGRATULATIONS!");
631     }
632 } else {
633     Serial.println("");
634     Serial.print("Next question: ");
635     Serial.println(qIndex + 1);
636     delay(1500);
637     showQuestion();
638 }
639 } else {
640     Serial.print(" - WRONG! (Correct: ");
641     Serial.print(correct);
642     Serial.print(" = ");
643     Serial.print(audioFile);
644     Serial.print(" - 4)");
645     Serial.println("");
646
647     playIncorrectTone();
648
649     counter = 0;
650     lcdCounter();
651     Serial.println("Try again...");
652     delay(1500);
653     showQuestion();
654 }
655 }
656 |
657 lastCountState = countState;
658 lastNextState = nextState;
659 lastResetState = resetState;
660 }
661 }
662
663 // ----- WIN ----- //
664 else if(state == WIN) {
665

```



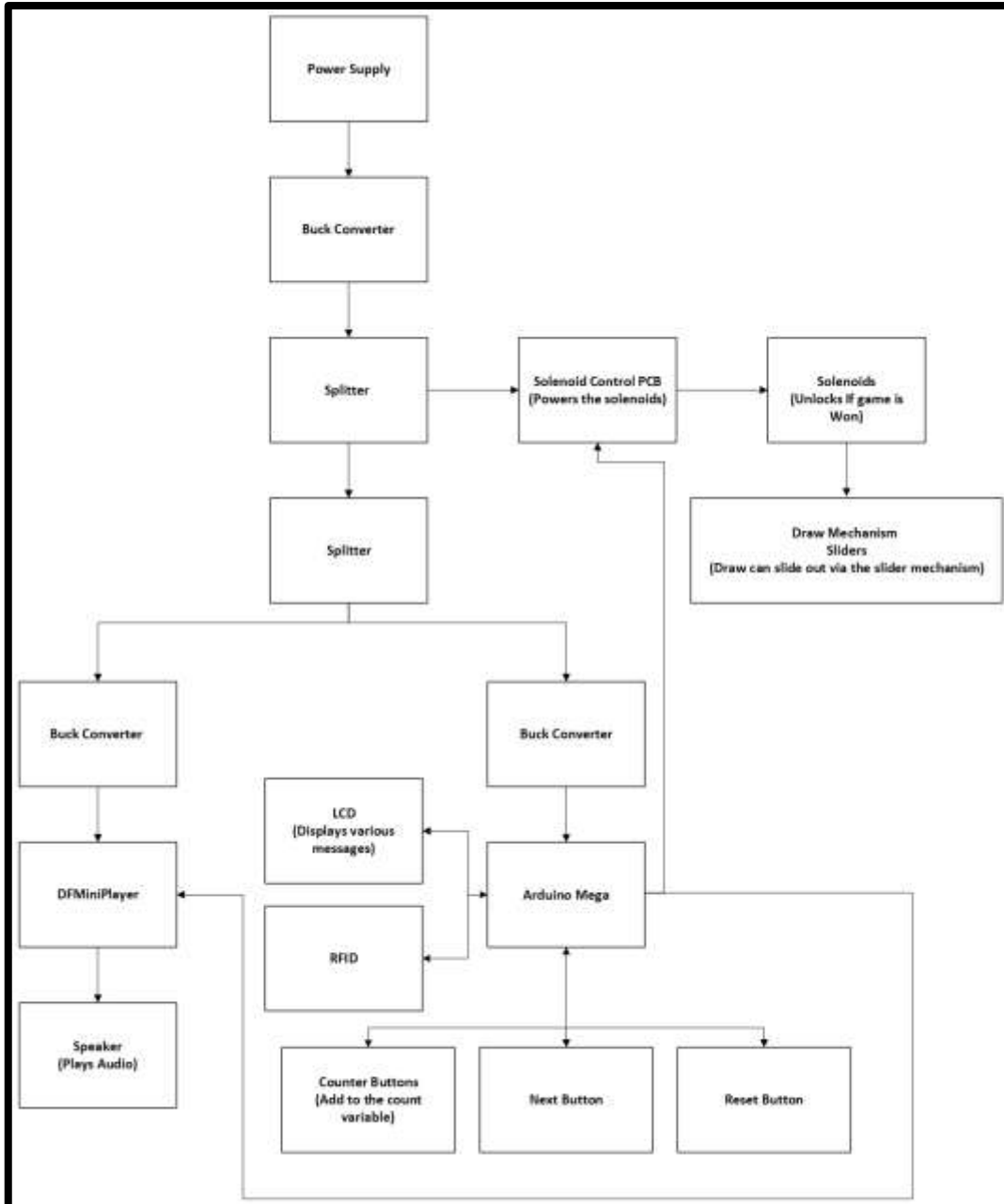
```

666     if (now - winStartTime >= solenoidDuration) {
667         digitalWrite(SOL1, LOW);
668         digitalWrite(SOL2, LOW);
669     }
670
671
672     unsigned long elapsed = now - winStartTime;
673     if (elapsed <= winTotalDuration) {
674         setRainbowColor(elapsed, winTotalDuration);
675     }
676
677
678     if (elapsed >= winTotalDuration) {
679         state = WAIT_CARD;
680
681         if(lcdWorking) {
682             showScanMessage();
683         }
684
685         shuffleQ();
686         Serial.println("");
687         Serial.println("=====");
688         Serial.println("Game reset - Scan card to play again");
689         Serial.println("=====");
690
691         delay(1000);
692         playWelcomeMessage();
693     }
694 }
695

```

This final chunk of code resets the everything for the next game

Final Block Systems Diagram



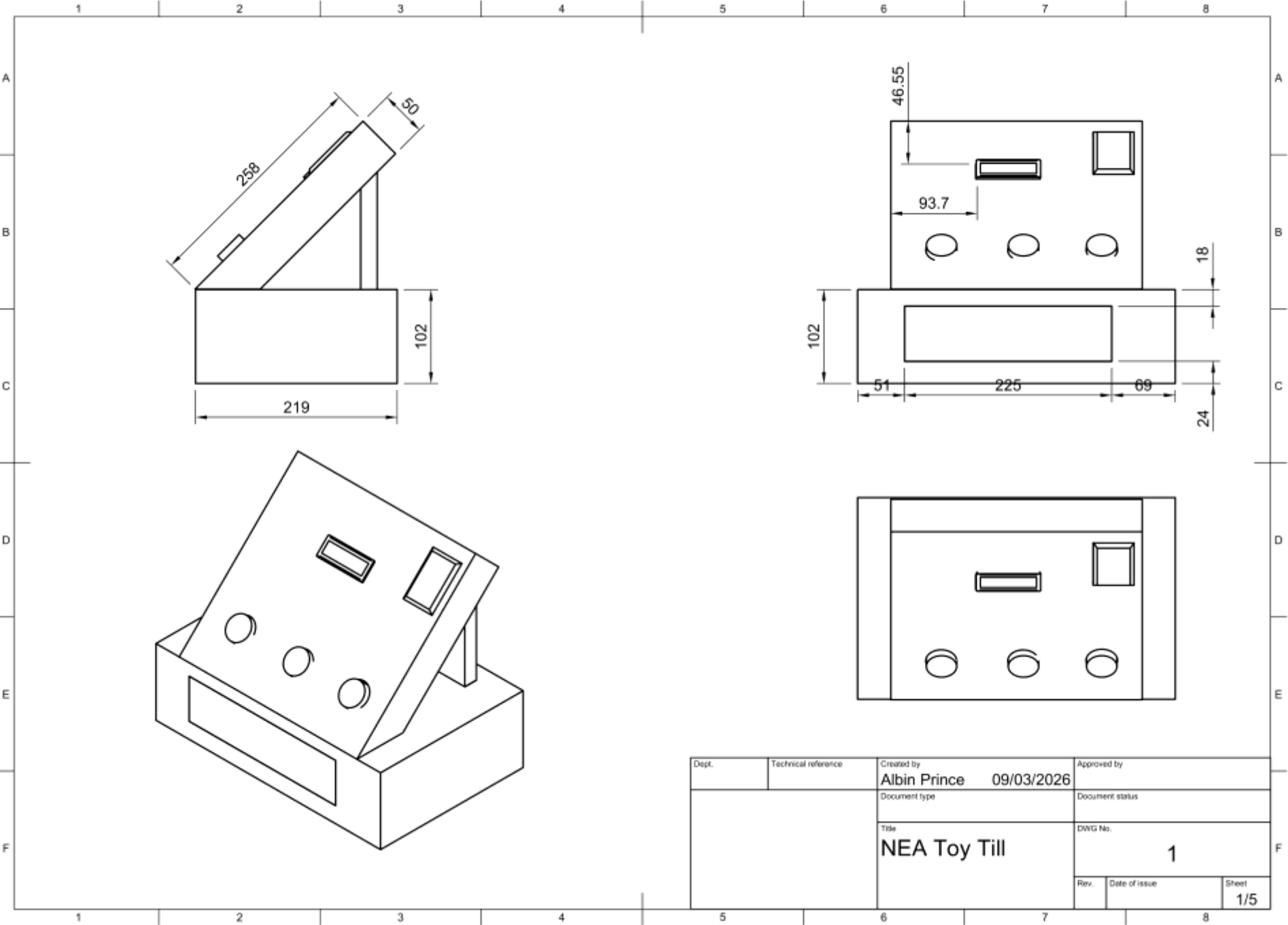
1. Wait for a valid RFID card to be scanned to start the game.
2. Once the game starts, a random order of 10 fruit questions is created and stored in an array called question Order.
3. For each question, store the correct number of fruits (1–10) in a variable called target quantity.
4. The LCD must display the question, for example: *“How many apples?”*
5. The LCD must show the current counter value in the largest possible format.
6. A single press on button 1 must increase the counter by 1 each time it is pressed.
The LCD updates every time the value changes.
7. A press on button 3 must reset the counter back to 0.
8. When button 2 is pressed, the value in counter is compared to target quantity.
9. If the numbers match, the LCD should show “Correct!” for a short time before moving to the next question.
10. If the numbers do not match, the LCD should show “Try Again” and the counter must reset to 0.
11. When all 10 questions are answered correctly, the LCD shows *“Well Done! You Win!”*
Both solenoids must activate for 3 seconds.
12. The game then resets automatically, and a new random set of questions is generated.
13. All variable changes must be printed to the Serial Monitor, including:
 - current fruit
 - counter value
 - correct/incorrect answer
 - RFID UID scanned
 - question progress



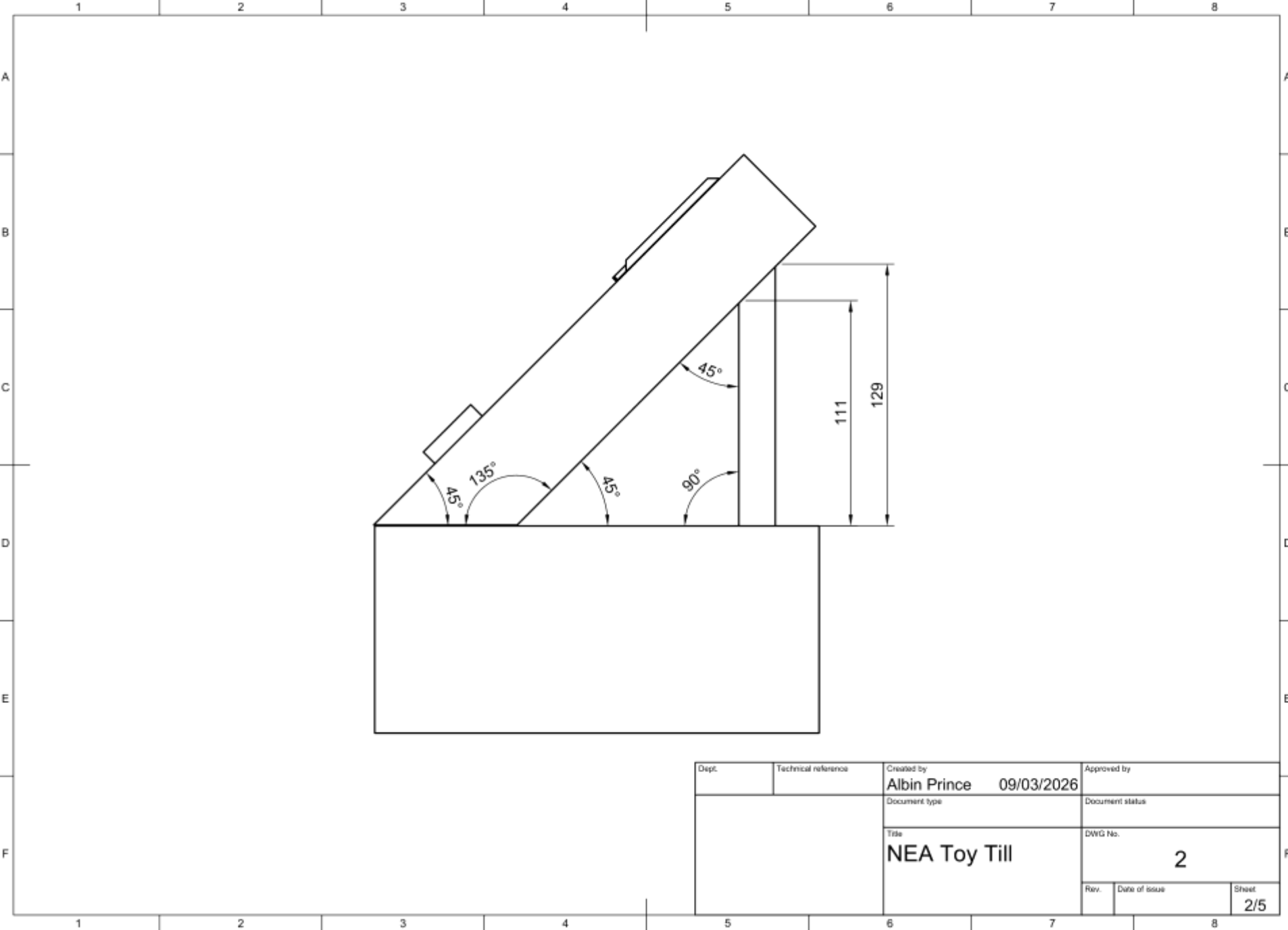
Final System Drawings



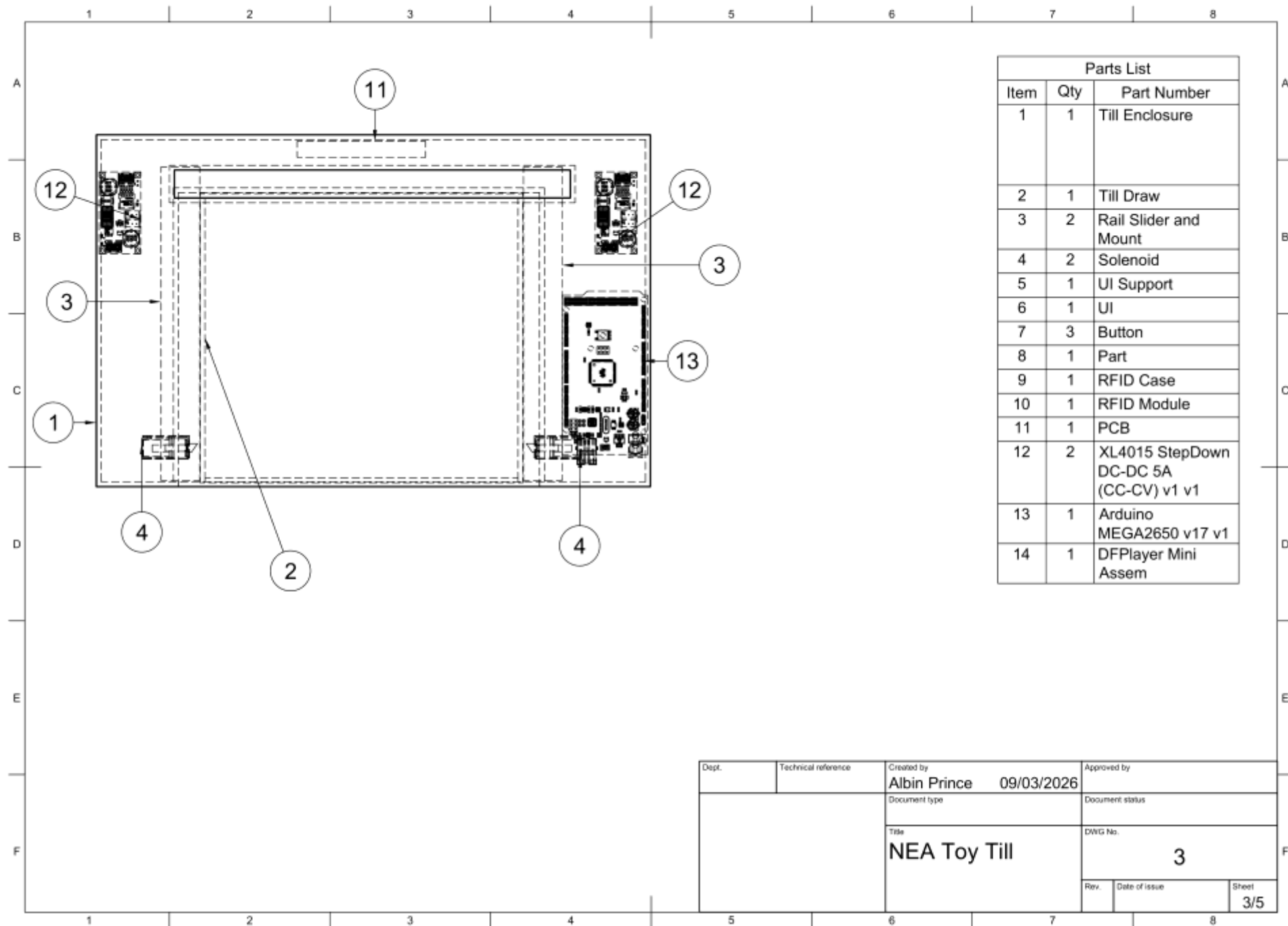
Overall Dimensions



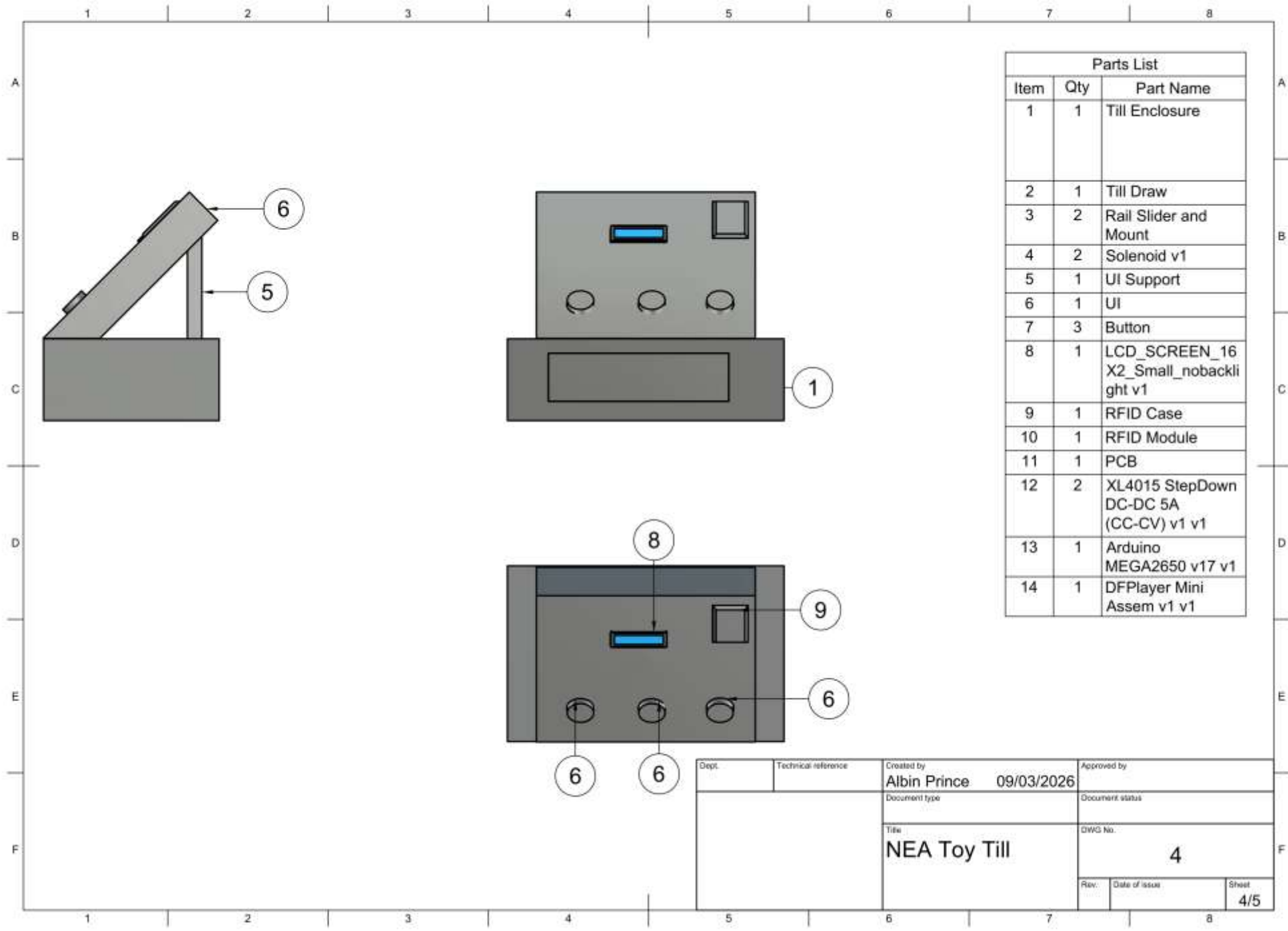
User Interface Angles



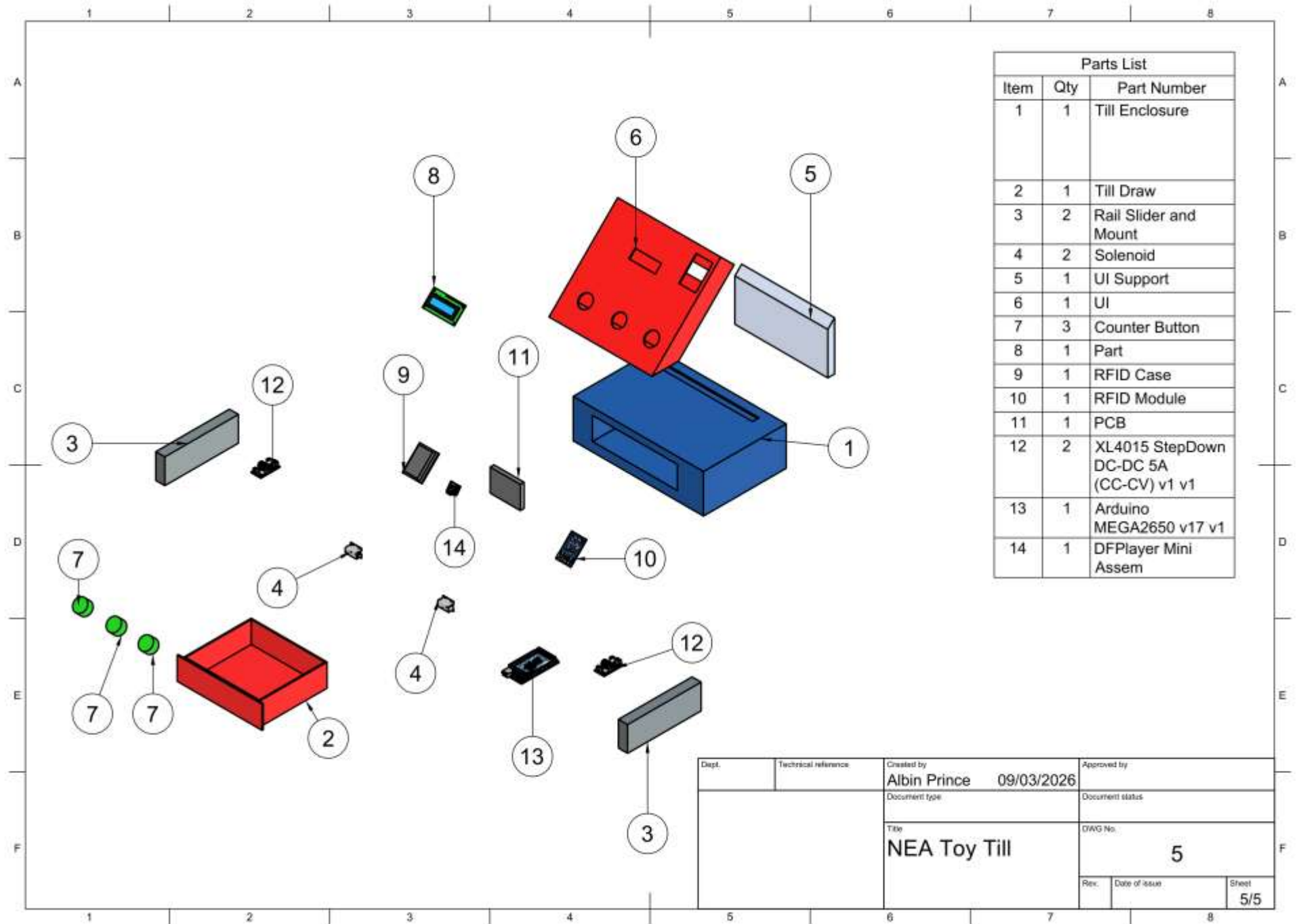
Top View of the internal components and draw



Externally
accessible
components



Exploded View of all the Components





Production



Manufacturing Plan

Stages of Build	Time to Build	Processes and materials	Tools and equipment	Quality control checks & expected tolerances	Repeatability	Health and Safety	Alternations to original design
Making a virtual model of the circuit	2 hrs	I will use the Fritzing software to make model of the circuit with all the modules connected correctly	Computer Fritzing application	I check the entire circuit would work before building it physically	It is very repeatable as the files are saved on the application's cloud and on my computer	Taking regular breaks to avoid damage to the eyes Take a 5 mins break every 30 minutes	
Modeling a custom PCB for the solenoids	30 mins	I will use Techsoft PCB software to make a PCB for the solenoid activation circuit	Computer Techsoft PCB Design Software	It must be a maximum of ± 1 mm off for the exact dimensions as it will be going inside the base of the toy till behind the draw.	It is very repeatable as the files are saved on a computer	Taking regular breaks to avoid damage to the eyes Take a 5 mins break every 30 minutes	
Printing off the PCB Design on acetate	10 mins	I will use the school printers to print of my PCB design on a sheet of acetate	Computer Printer Techsoft PCB Design Software	It has to print in high contrast black ink where the tracks are so there are no issues with the PCB's track conductivity	It is very repeatable as the files are saved on a computer Make multiple copies of the PCB on the same acetate	Taking regular breaks to avoid damage to the eyes Take a 5 mins break every 30 minutes	
Exposing the PCB Design	5 mins	I will use a light box to expose the PCB to the design on the acetate	Lightbox	It must be exposed for the right amount of time. If it is exposed for too little time, then it under develops. If exposed for too much then, the light gets under the tracks	Make multiple copies of the PCB on the same board for mass manufacturing	UV googles to protect against eye damage due to direct UV light	

Stages of Build	Time to Build	Processes and materials	Tools and equipment	Quality control checks & expected tolerances	Repeatability	Health and Safety	Alternations to original design
Developing the PCB	3 mins	I will use a developing solution to develop the PCB	Developer solution Water Bowl	Visual Inspection – any cloudy remains means that it is underdeveloped. Any tracks and writing on the PCB must have a sharp definition	Easily repeatable if using the same developer's solution if otherwise then have to adjust the length of time left in the solution	Safety googles and gloves Ventilated area	
Etching the PCB	0.5 hrs	I will use the school's Ferric Chloride tank to etch the PCB and form the tracks	Ferric Chloride tank	Visual Inspection – Any tracks and writing on the PCB must have a sharp definition	Easily repeatable if you have developed the PCB Etch multiple copies of the PCB on the same board	Safety googles, apron and gloves Ventilated area	
Coding the program	5hr	I will use online guides and my own coding knowledge to code the program in the Arduino IDE	Computer Arduino IDE USB – B cable	It must meet my program specification	It is hard to repeat as coding an entire game from scratch with little to no coding knowledge is hard	Taking regular breaks to avoid damage to the eyes Take a 5 mins break every 30 minutes	I added difficulty levels to further offer more options to play the game
Building the Circuit	1hr	I will use jumper wires and my virtual model of the circuit to build the physical circuit	Computer Fritzing Components Screwdriver	It must match the virtual model of the circuit	It is very repeatable if you have the components and wires Use templates to create wiring of correct lengths	Be cautious of what wire plug in where	
Model the Toy Till	2hr	I will use the Fusion 360 software to make the model of the base of the toy till	Computer Fusion 360	It must be the exact dimensions of the base	It is very repeatable as the files are saved on the application's cloud and on my computer	Taking regular breaks to avoid damage to the eyes Take a 5 mins break every 30 minutes	I made changes to the placement of the components. This meant that it is more organized and easier to manage

Stages of Build	Time to Build	Processes and materials	Tools and equipment	Quality control checks & expected tolerances	Repeatability	Health and Safety	Alternations to original design
Designing the Toy Till	2hr	I will TechSoft 2D Design software to design my toy based off the 3d model	Computer TechSoft 2D Design	It must be exactly the right dimension	It is very repeatable as the files are saved on my computer	Taking regular breaks to avoid damage to the eyes Take a 5 mins break every 30 minutes	
Glueing the toy till	2hr	I will use the laser cut parts and PVA glue to stick and make the till	Laser-cut parts PVA Glue	All the parts must be glued in their respective right spaces	It is fairly repeatably if you have the laser-cut parts using a jig to ensure perfect alignment of components whilst gluing	Gloves Apron Ventilated area	
Painting the toy till	3hr	I will use the parts of the till and acrylic paint. I painted using sponges for coating and paint brush for finer details. All the parts were double coated with white primer, then double coated with red or blue paint	Acrylic paint Sponges Paint Brush	The paint job must look nice, so it seems appealing to children	It is somewhat repeatable. This is because if depends on an individual's skills to paint and shade and type of paint used Use a hanging rail and spray booth	Gloves Apron	
Fitting the electronics	30 mins	I will disassemble the circuit and reassemble it inside the toy till	Components Screwdriver Glue	It must match the virtual model of the circuit	It is very repeatable if you have the components and wires	Be cautious of what wire plug in where	
Final Check	N/A	I will play the game to make sure the toy performs as expected. If it doesn't behave as intended then I will debug the issue and solve it	Computer Arduino IDE	It must perform as expected`	N/A		

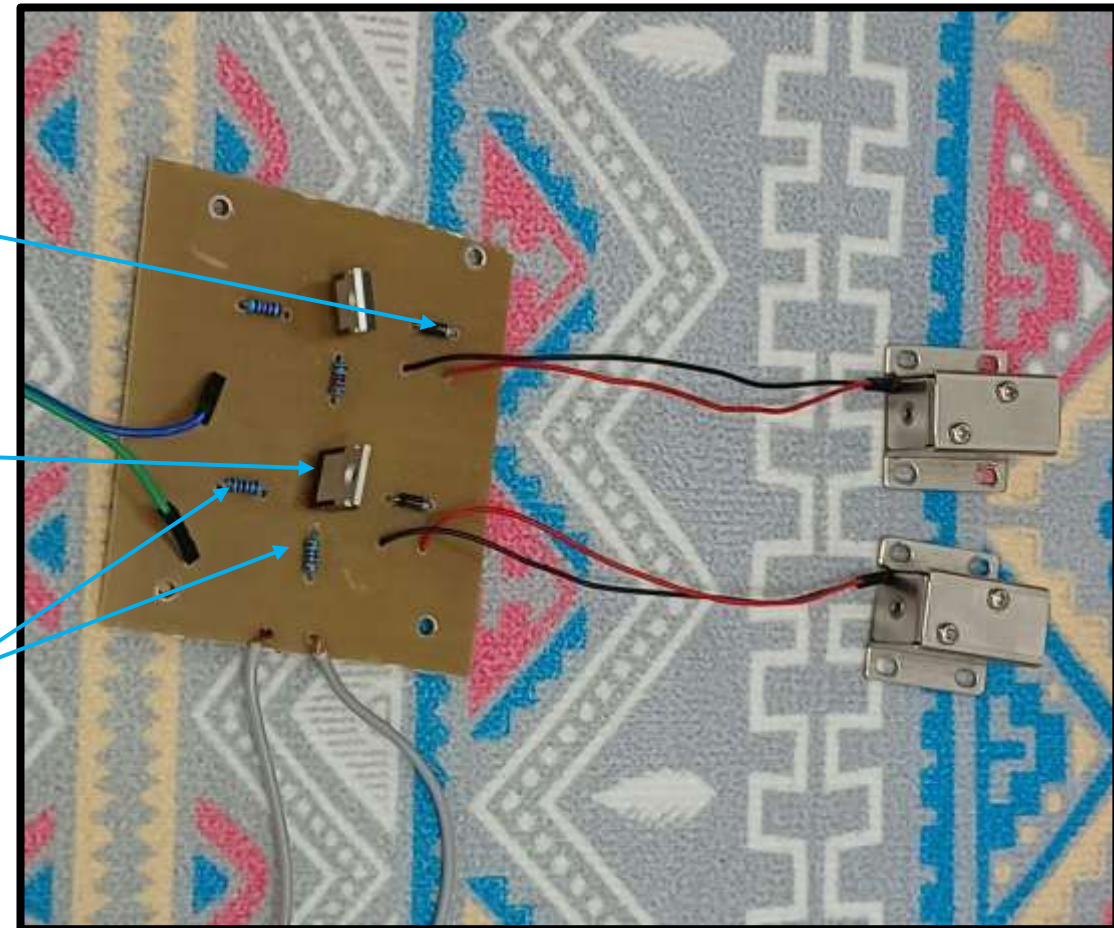
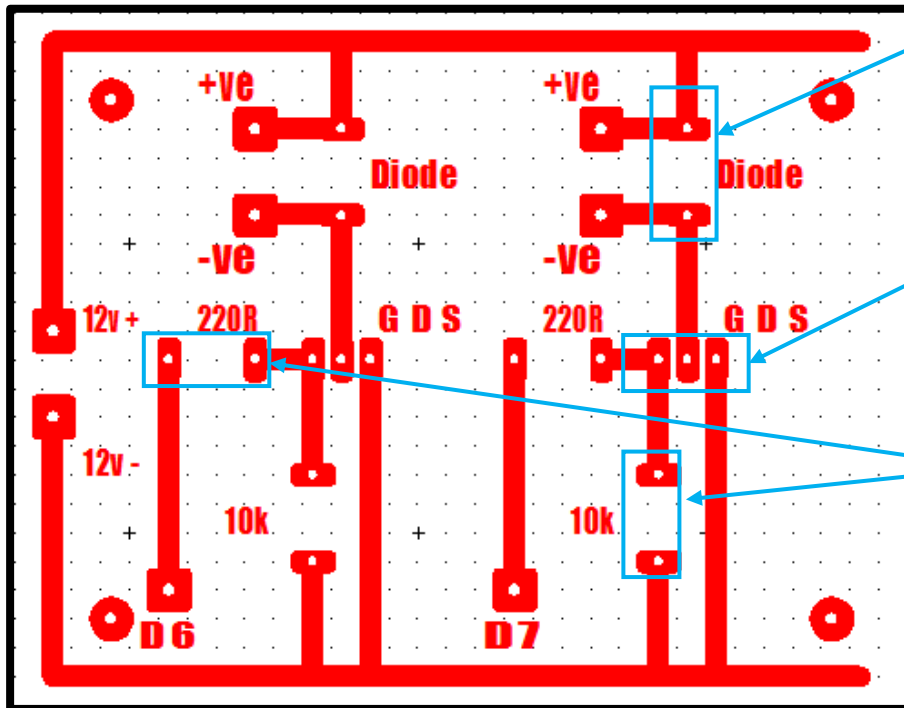


Manufacturing



Solenoid Control Board

To interface the low-current Arduino Mega (40mA per pin) with the high-demand 12 V solenoids (1.5A), I designed a custom PCB utilizing IRLZ44N Logic-Level MOSFETs as high-speed switches. These specific N-channel transistors were selected because they reach full saturation at a 5V gate voltage, ensuring minimal internal resistance and preventing overheating during operation. To ensure circuit reliability and protect the microcontroller, I integrated 10k Ω pull-down resistors to prevent floating gates during startup, 220 Ω gate resistors to limit current spikes, and flyback diodes across the solenoid terminals to dissipate back EMF voltage spikes. During the assembly stage, I prioritized high-quality solder joints to handle the 1.5A load and performed continuity testing with a multimeter to verify the integrity of the traces before applying power.



Main Chassis

To assemble the main chassis components, I utilized PVA (Polyvinyl Acetate) wood glue to bond the precision laser-cut finger joints (also known as comb joints). This joint type was selected because it significantly increases the surface area for the adhesive to bond, resulting in a much stronger mechanical connection than a simple butt joint.

During the gluing process, I applied thin, even layers of PVA to all mating surfaces to ensure maximum penetration into the MDF fibers. To achieve a high-quality finish, I used set squares to verify that all panels were perfectly perpendicular (90°) and used G-clamps to apply uniform pressure while the adhesive cured.

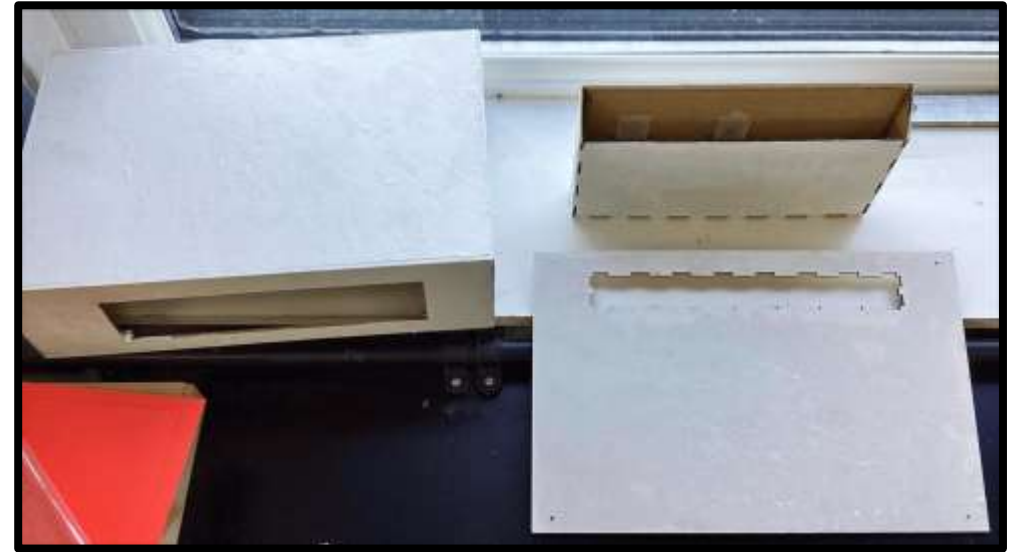
Any excess PVA was immediately removed with a damp cloth to prevent it from interfering with the final surface finish or the smooth operation of the sliding mechanisms.



Painting

I selected a high-contrast primary colour palette of red and blue specifically because the target users for this till are young children. Developmentally, primary colours are the easiest for children to identify and distinguish, which makes the toy more engaging and supports early cognitive development. By using high-saturation acrylics, I have created a playful, "toy-like" aesthetic that appeals to a younger demographic while clearly communicating the product's function as a role-play learning tool. This bold styling acts as a visual affordance, helping children recognize different interactive sections of the till and improving their hand-eye coordination during play.

For the surface treatment, I utilized water-based acrylic paint due to its fast-drying properties and high opacity, ensuring a vibrant finish that fully masks the white primer. Unlike oil-based alternatives, acrylic paint remains slightly flexible once dry, allowing the finish to withstand the natural expansion and contraction of the MDF substrate without cracking. To achieve a professional result, I used a sponge-stippling technique for the base coats to eliminate unsightly brush marks and ridges. This produced a consistent, matte surface texture that mimics the durable powder-coated plastic or metal finishes typically found on commercial point-of-sale hardware.



Solenoids

To secure the drawer, I used a combination of mechanical fasteners and chemical bonding to align the solenoid with a custom latch plate mounted on the drawer side. The solenoids are bolted onto threaded standoffs using M3 screws, providing a stable mounting point that was then bonded to the internal chassis using Cyanoacrylate (super glue). This "standoff" method was critical to ensure the solenoid was elevated to the exact height required for the spring-loaded plunger to engage with the latch. By using screws for the solenoid-to-standoff connection, I have ensured the system is serviceable, allowing for easy recalibration of the locking point if the drawer alignment shifts over time.

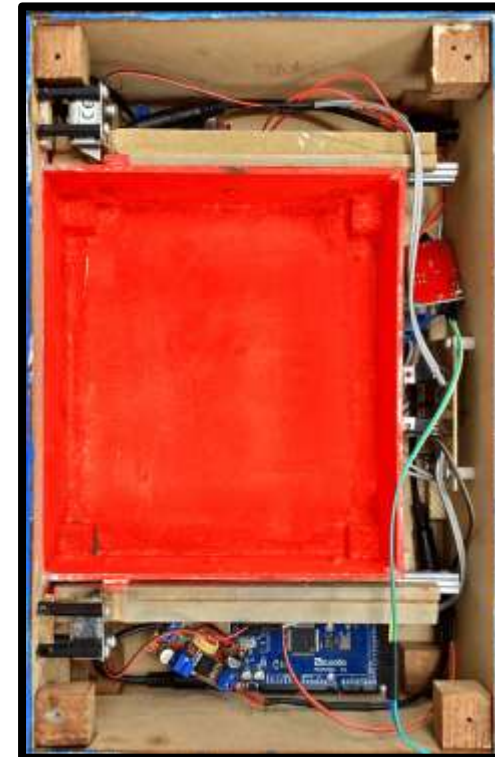
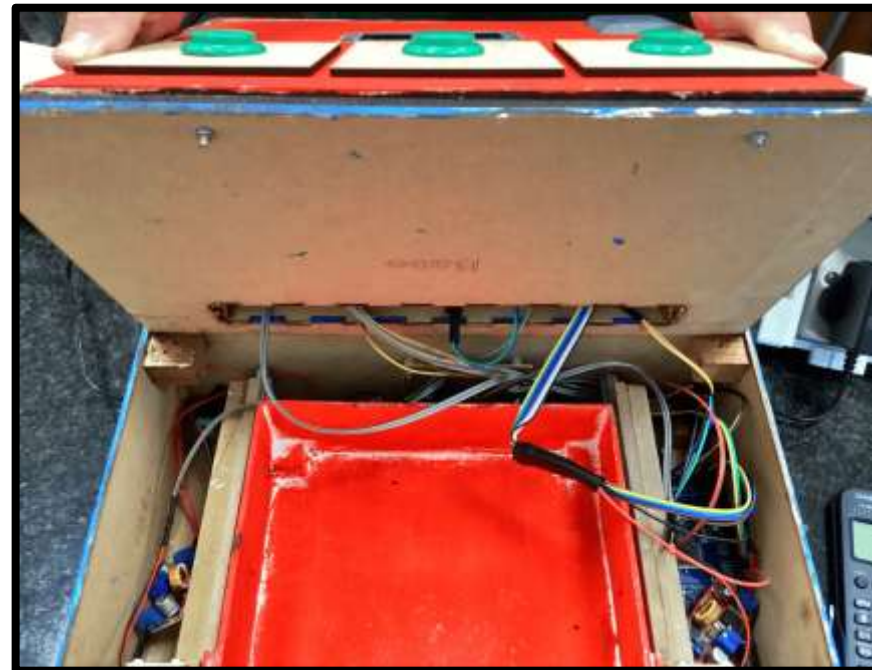
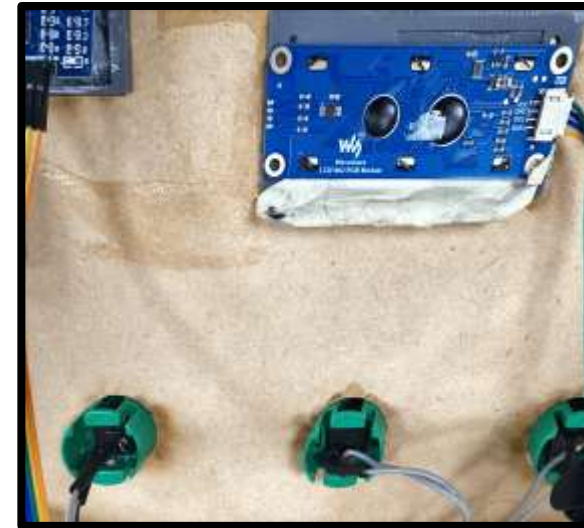
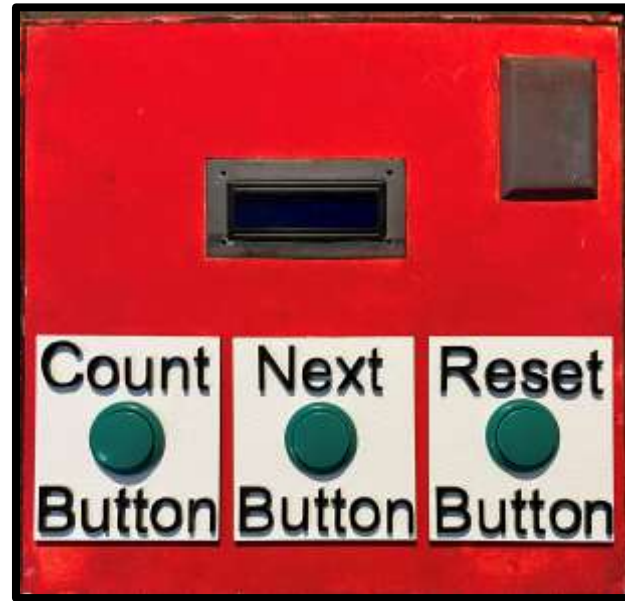
During the assembly of the latching mechanism, I performed rigorous clearance checks to ensure that the solenoid plunger could retract fully without snagging on the latch plate. This required precise measurement to ensure the "throw" of the solenoid was sufficient to clear the latch when the Arduino sends a 12v pulse. I also verified the structural integrity of the bonded standoffs, ensuring they could withstand the repetitive kinetic force and vibrations caused by the solenoid firing. This precise alignment between the fixed solenoid and the moving drawer latch is what allows the till to remain securely locked until the correct software trigger is received.



Merging the electronics and physical parts

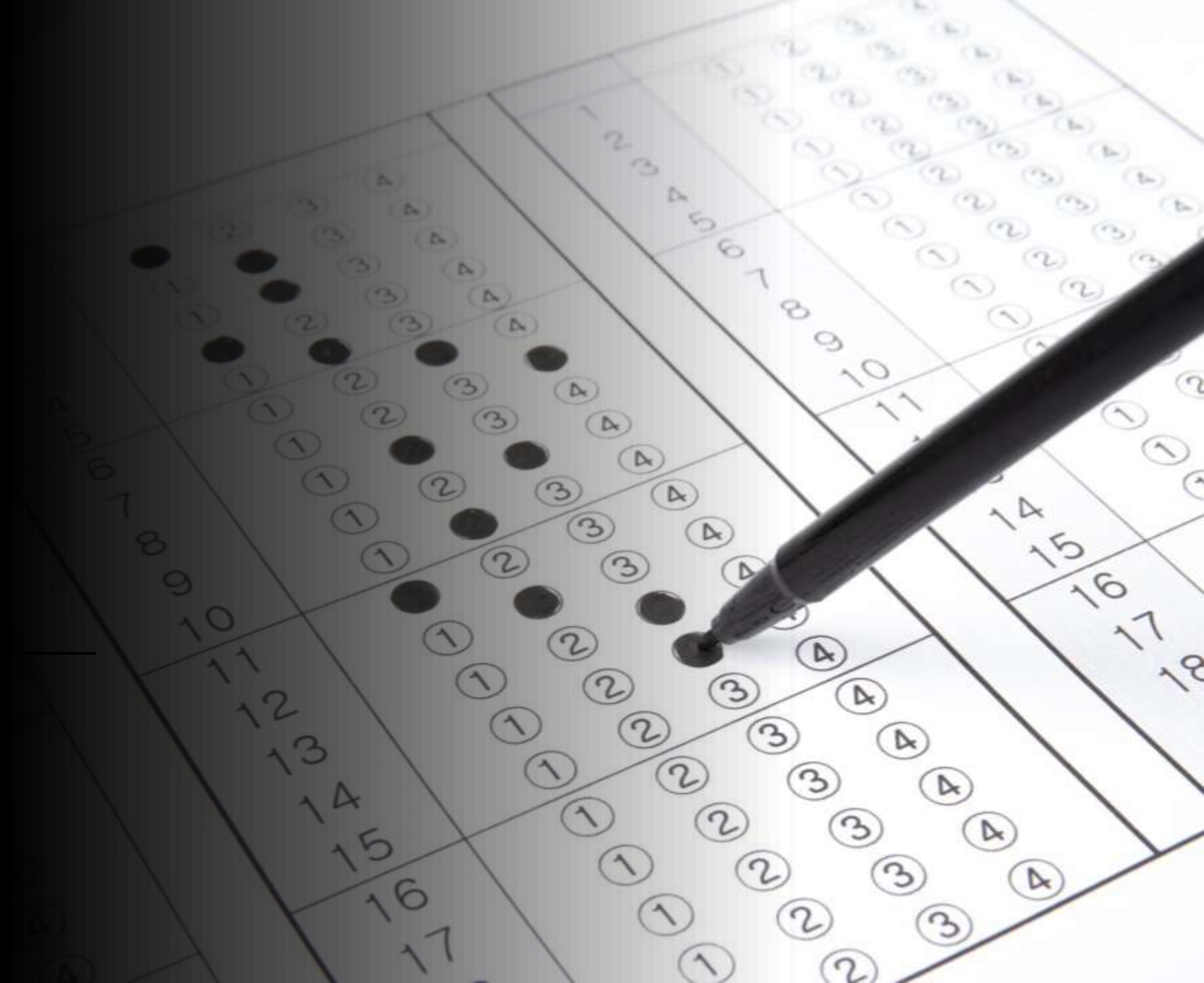
The final stage of production involved the systems integration of the electronic control suite into the chassis. I mounted the I2C LCD, the three primary control buttons (Count, Next, and Reset), and the RFID module into precision laser-cut apertures. These arcade-style buttons were selected for their tactile feedback and high durability, both of which are essential for a product intended for children. To ensure the internal components remain secure during operation, I utilized plastic standoffs and adhesive mounts for the Arduino Mega and the custom solenoid control board. This prevents the delicate circuitry from shifting or shorting against other conductive parts. The top panel was designed as a removable maintenance hatch, allowing for easy access to the internal wiring while ensuring the "live" electronics remain safely enclosed and inaccessible to the end user during play.

Given the complexity of the wiring harness, I prioritized cable management to prevent interference with the mechanical sliding of the drawer. I used ribbon cables and cable ties to bundle the signal wires from the buttons and LCD, ensuring they were routed away from the high-current 12v lines powering the solenoids. Before the final "close-up" of the chassis, I conducted a system-wide diagnostic test: verifying that each button press correctly updated the LCD logic and that the solenoids fired with the correct timing to release the drawer latch. This methodical approach to final assembly ensures that the toy is not only functional but also safe and robust enough for repetitive play.





Evaluation



Tolerances

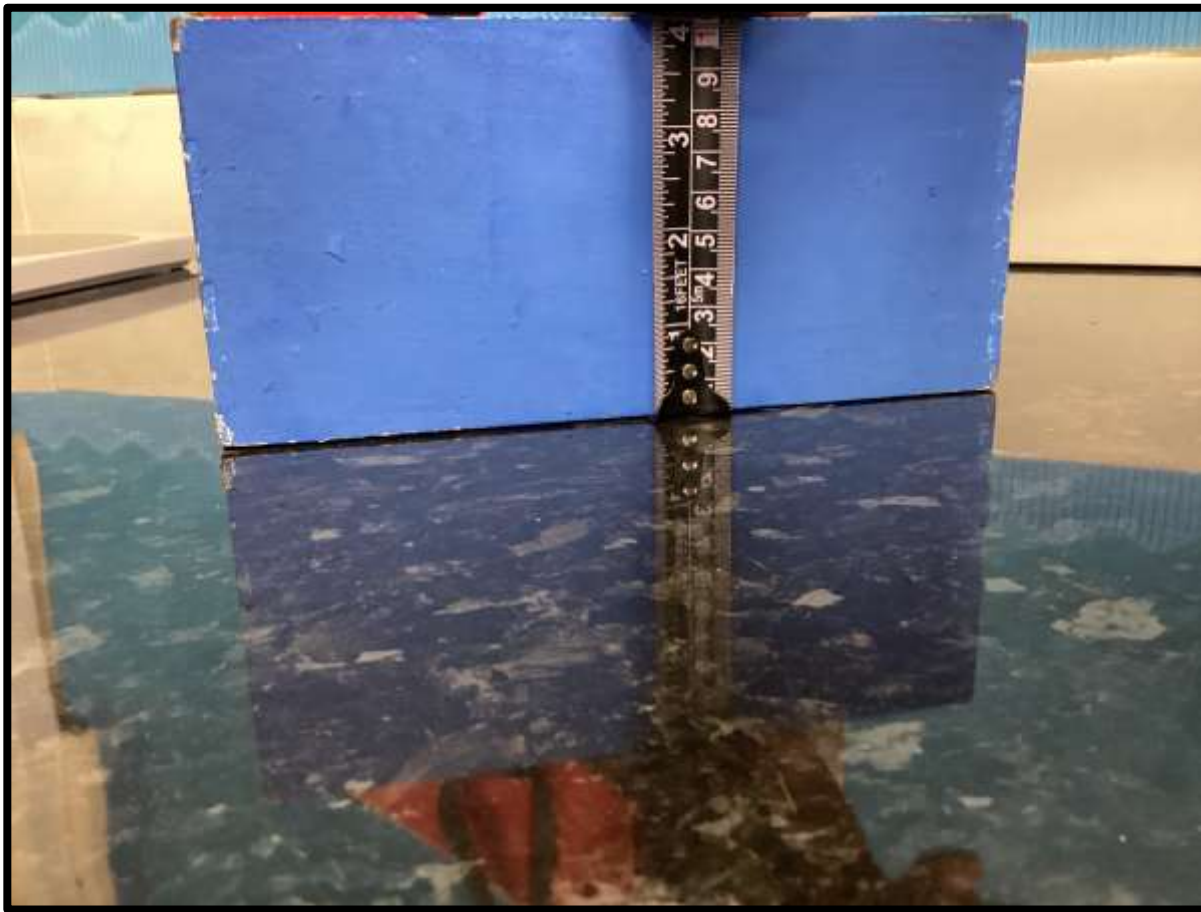
Part	Expected Measurement	Tolerances	Actual Measurement	Measurement Device	Pass / Fail
Length of Till	345 mm	± 6mm	345mm	Measuring Tape	Pass
Depth of Till	219 mm	± 6mm	219mm	Measuring Tape	Pass
Height of Till	102 mm	± 6mm	102mm	Measuring Tape	Pass
Thickness of Panel	3 mm	± 0.1mm	3.01mm	Vernier Callipers	Pass
Height of Draw	60 mm	± 0.1mm	60 mm	Measuring Tape	Pass
Width of Draw	225 mm	± 0.1mm	224.9 mm	Measuring Tape	Pass
Height of UI Support (Short)	111 mm	± 0.1mm	111 mm	Measuring Tape	Pass
Height of UI Support (Long)	129 mm	± 0.1mm	129 mm	Measuring Tape	Pass
Height of UI	258 mm	± 0.1mm	258 mm	Measuring Tape	Pass
Width of UI	273 mm	± 0.1mm	273 mm	Measuring Tape	Pass
Depth of UI	50 mm	± 0.1mm	50 mm	Measuring Tape	Pass



Length of the Till – 345mm



Depth of the Till – 219mm



Height of the Till – 102mm



Thickness of the Panels – 3.01mm

Evaluation against Specification

Specification	Requirement	Method of testing	Results	Pass / Fail
Weight of Till	1500 g < Weight > 3000 g	Weighing Scale	2393 g	Pass
Size of letter in LCD	Letter < 3 mm	Ruler	3.5 mm	Pass
Button Width	Width > 15 mm	Venier Callipers	24.5 mm	Pass
Button Spacing	Spacing > 20 mm	Venier Callipers	52.5 mm	Pass
Dimensions of Separate Parts	Length > 30 mm Height > 30 mm	Ruler	50 mm by 77 mm	Pass
Audio Volume	Volume < 70 dB	Decibel Meter	60 dB	Pass
Audio Frequency Range	80 Hz < Audio < 12 kHz	Frequency Spectrum Meter	100 Hz to 10 kHz	Pass
Start-up Time	Time < 5 seconds	Stopwatch	3.58 seconds	Pass
UI Angle	44° < Angle < 54°	Protractor	45°	Pass
Force to open Draw	10N	Newton meter	1.6 N	Pass
Force to press buttons	1N	Weights	1 N	Pass

Serial	Specification	Description of testing	Result of testing
1	Total Mass of the product must be less than 3kg but more than 1.5kg	I used a weighing scale to measure the mass of the product	Pass : The product weighed in at 2393g so roughly 2.4kg
2	The letter sizes on the display must be more than 3mm to make it easier for the child to see	I used a ruler to measure the size of the LCD character spaces	Pass : The size was roughly 3.5mm

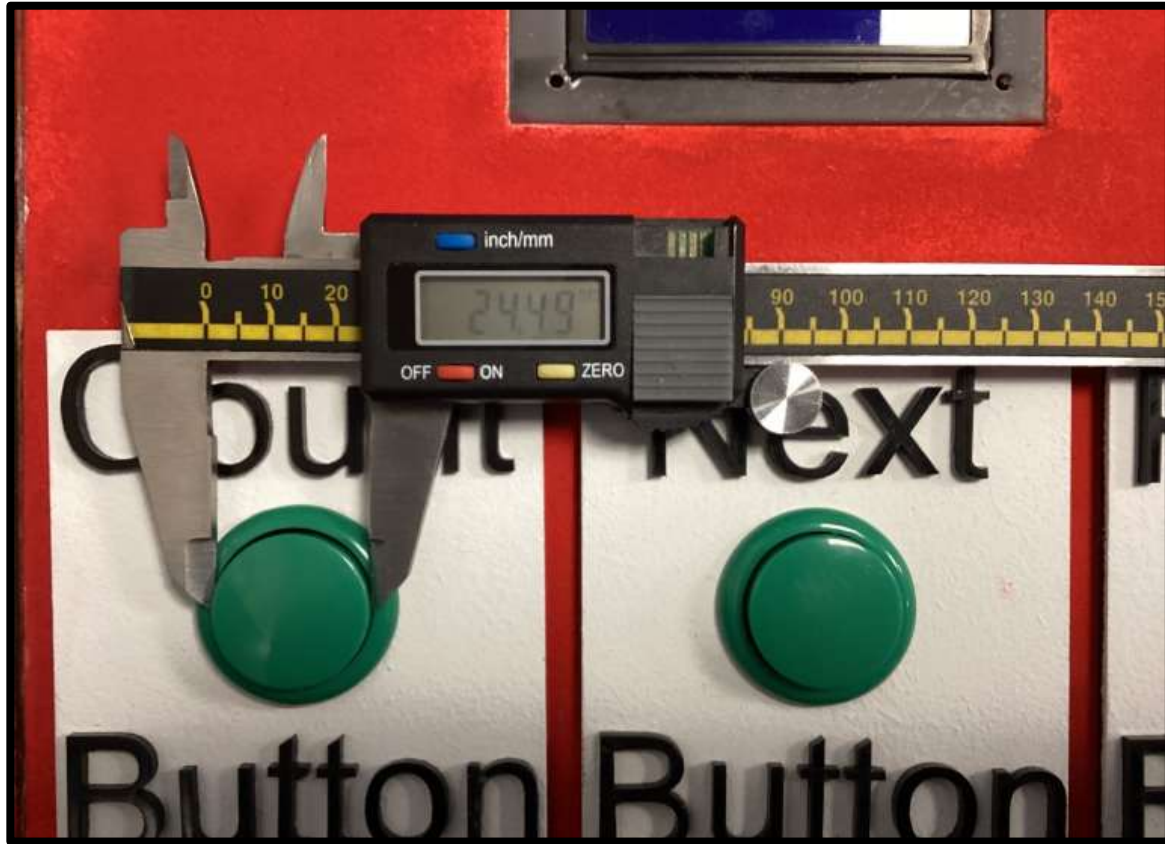


Weight of Till - 2393g



Size of letters in LCD – 3.5mm

Serial	Specification	Description of testing	Result of testing
3	The Buttons must be larger than 15mm. So it is be easy to use by the child	I uses a Venier Callipers to measure the diameter of pressable parts of the button	Pass: The diameter of the button is roughly 24.5mm
4	The Button Spacing must be wider than 20mm. It is to avoid accidental presses to 2 buttons at the same time	I uses a Venier Callipers to measure the distance between the pressable parts of the button	Pass: The distance between the buttons was roughly 52.5mm



Button Width – 24.49mm



Button spacing – 52.54mm

Serial	Specification	Description of testing	Result of testing
5	All Parts must be larger than 30mm by 30mm It must no be a choking hazard to the child	The dimensions were measured by a ruler	Pass: It is roughly 77mm by 50mm



Parts Height– 50mm



Parts Length – 77mm



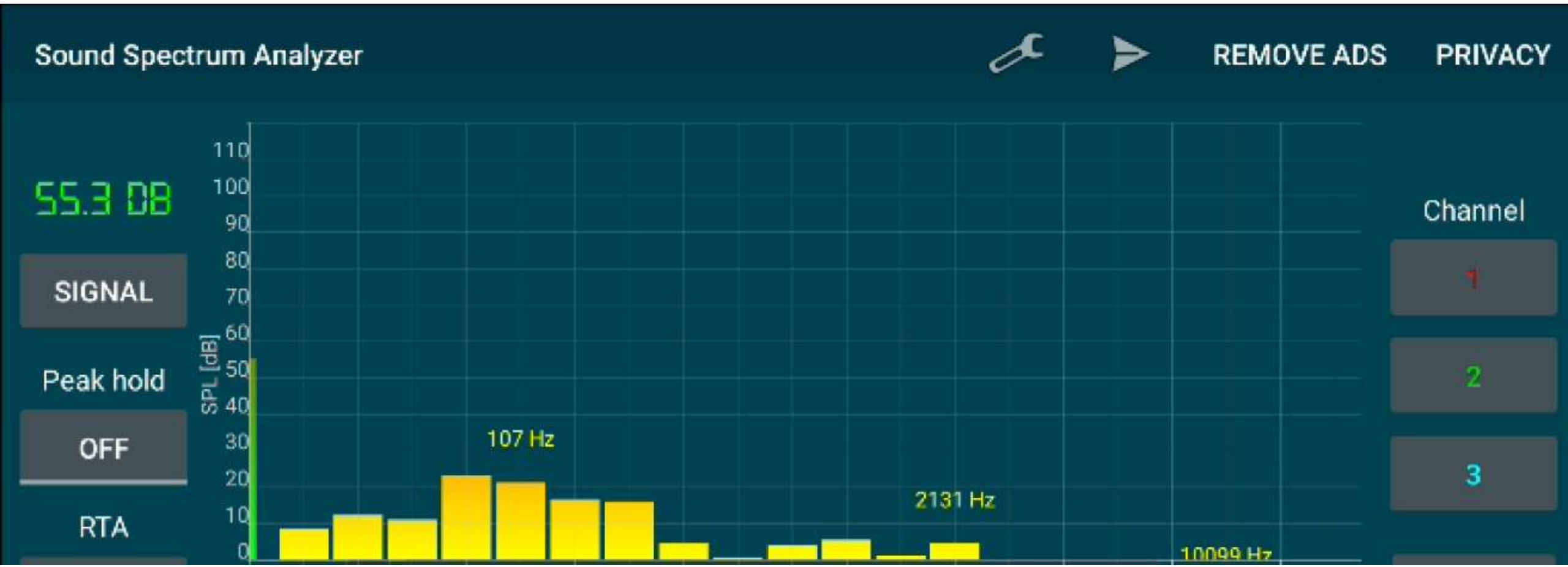
Serial	Specification	Description of testing	Result of testing
6	Audio Volume must be less than 70 decibels. It must be at a safe volume and not damage the child's ear	I used a decibel meter to see the maximum volume it reaches	Pass: The audio was roughly at about 60 dB.

Sound from the toy

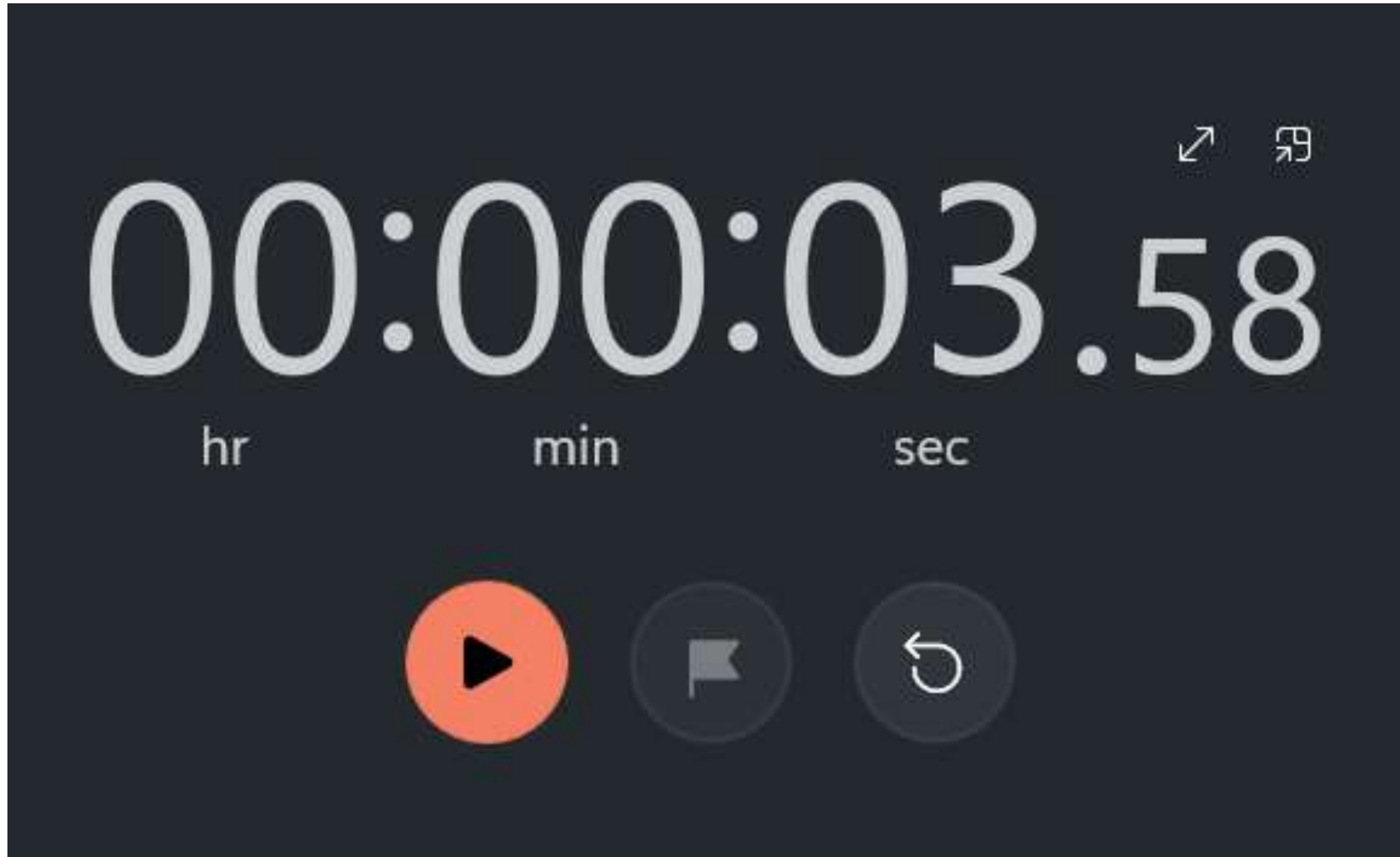
Average sound reached at 60
Decibels

Serial	Specification	Description of testing	Result of testing
7	The audio frequency range must be from 80 Hz – 12,000 Hz. It must be audible to children while not damaging their auditory ability	I used a audio spectrum analyser to analyse and find the audio range that is produced	Pass: Most of the audio from the toy was roughly between 100hz to 10000hz.

Sound frequency
Average from 100 Hz all the way to 10 kHz

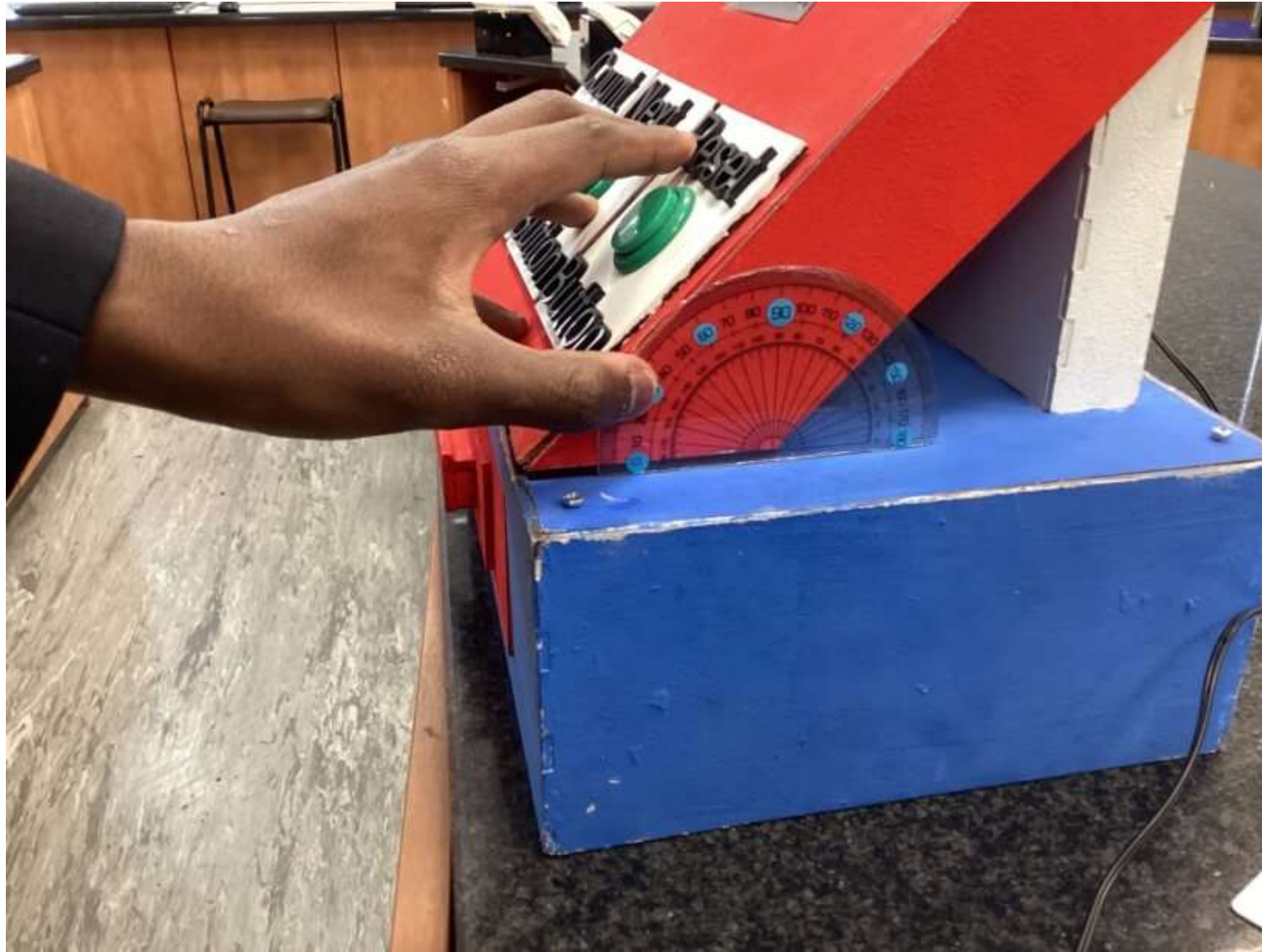


Serial	Specification	Description of testing	Result of testing
8	The Start-up Time must take less than 5 second. It must be quick to load to maintain engagement from the child	I used a stopwatch to measure how quickly it loads	Pass: I took roughly 3.5 seconds to load



Time : 3.58 seconds

Serial	Specification	Description of testing	Result of testing
8	The user interface must be larger than 44° but less than 54° This is because the user need to be able to comfortably play with the toy by keeping it at an optimum angle	I used a protractor to measure the angle at which the UI is mounted at	Pass : The angle was 45°



Angle : 45°

Serial	Specification	Description of testing	Result of testing
9	The force to open the draw must take less than 10N of force to open. It must be easy to open for a child	I used a newton metre to measure the force required	Pass: The force required to open the draw was 1.6 N



Force = 1.6 N

Serial	Specification	Description of testing	Result of testing
10	The force to press the button must take less than 1N of force to open. It must be easily pressable by the child to avoid fatigue	I used a weights to measure the force required	Pass: The force required to press down the buttons is 100g which is exactly 1 N



Force = 1 N

Operation of my product



Evaluation : System Operation

Analysis:

Overall, all aspects of my fruit counting game work together effectively to create an engaging and intuitive educational experience, thus meeting the intended learning outcome.

The RFID authentication ensures that only the authorised card can start a game, providing a clear security boundary before gameplay begins. Once a valid card is tapped, the difficulty selection screen appears promptly, allowing the user to choose Easy, Medium, or Hard using the three buttons—each with a distinct function that is immediately understandable. The game's core mechanic of counting fruit pieces using a dedicated count button is responsive and free from bounce, and the answer is submitted with a separate “next” button. Audio feedback for correct and incorrect answers is delivered through the DFPlayer Mini, which plays clear sound effects and question prompts, reinforcing the learning process. The LCD screen guides the user at every stage: a blue “Scan a card” screen, a green “Access Granted” confirmation, a yellow difficulty menu, a dynamic loading bar, and a vibrant rainbow animation upon winning—all of which enhance visual engagement. Solenoids are triggered at the win condition, providing a tangible reward that lasts for three seconds while the win message remains on screen for a total of six seconds.

An early issue where the “tap the card” audio played twice consecutively at startup was resolved by adding a cooldown timer and careful resetting of the welcome message timer, demonstrating the iterative refinement of the system. The layout and component choices—such as the Arduino Mega, MFRC522 RFID reader, DFPlayer Mini, and Waveshare LCD—mirror a scalable design; with appropriate power and relay modules, the solenoids could be upgraded to full-size actuators, making the prototype a solid foundation for a larger installation.

Improvements :

Whilst my fruit counting game functions reliably, several improvements could enhance its suitability as a complete product.

The solenoids are currently secured using threaded standoffs bonded to the chassis with cyanoacrylate. While this provides a stable mounting point, a more permanent solution would involve embedding threaded inserts directly into the chassis, eliminating any risk of adhesive degradation over time. The main chassis is assembled with PVA glue on finger joints, which offers strong adhesion, but for a production model, using a combination of support blocks and industrial grade thermosetting resin would increase structural durability.

The custom MOSFET driver board reliably switches the 12 V solenoids but integrating a separate power supply with built-in overcurrent protection would improve safety and reliability for long-term use. Moreover, adding a persistent high-score display (e.g., fastest completion time or streak count) would introduce a competitive element and encourage repeated play. The DFPlayerMini provides clear audio, but the volume level is fixed; incorporating a potentiometer or digital volume control would allow the user to adjust it to their environment.

The three physical buttons are intuitive, but their placement could be improved by integrating them into a single panel with clear labelling and tactile differentiation (e.g., different colours or shapes) to reduce the learning curve for new users. Finally, the solenoid reward currently triggers a simple 3-second activation; adding a more varied feedback sequence (e.g., a short animation on the LCD) would make the win condition feel even more satisfying.

Overall

With these refinements, the prototype would transition from a functional demonstrator to a polished, user-friendly educational tool suitable for extended use.

Evaluation : Manufacture

Analysis:

Overall, the manufacturing of my fruit counting game proceeded efficiently. A detailed production plan allowed me to follow a logical assembly sequence, minimising errors and rework.

The enclosure, component mounts, and internal supports were designed using TechSoft 2D CAD software and laser-cut from 3mm MDF. This process ensured high precision and repeatability – the same CAD files could produce multiple identical units. MDF proved easy to work with while offering sufficient rigidity to support the solenoids and electronics. The chassis was assembled using PVA wood glue on laser-cut finger joints. This joint type maximises the adhesive surface area, resulting in strong, durable connections that resisted movement during solenoid activation.

The most complex manufacturing task was the electronic sub-assembly. I designed a custom PCB using IRLZ44N MOSFETs to switch the 12V solenoids safely. The board was hand-soldered, with 10k Ω pull-down resistors, 220 Ω gate resistors, and flyback diodes to protect the Arduino. High-quality solder joints and continuity testing with a multimixer ensured reliable operation under a 1.5A load. The solenoids were mounted using threaded brass standoffs bonded to the chassis with cyanoacrylate, then secured with M3 screws. This “standoff” method raised the solenoids to the exact height needed for the plunger to engage the latch, and the screw connection made the system serviceable.

Quality control checks were performed throughout manufacturing. All laser-cut dimensions conformed to expected tolerances, every component fitted correctly, and all glued finger joints remained secure during repeated testing. However, the paint finish on the exterior enclosure was poor due to my limited painting skills, resulting in an uneven, brush-marked appearance.

Improvements :

Whilst the prototype functions reliably, several manufacturing improvements would enhance its quality and durability.

Paint Finish – The most obvious improvement is applying a higher-quality finish. Using spray paint with a suitable primer, applying multiple thin coats. Alternatively, coloured acrylic sheet or vinyl wrap could be used instead of paint.

Solenoid Mounting – The current standoffs bonded with cyanoacrylate are stable, but for long-term durability, mechanical fastening is preferable. Embedding threaded inserts directly into the chassis or using through-bolts with nuts would create a permanent, serviceable connection that cannot degrade over time.

Enclosure Assembly – While PVA glue on finger joints is strong, adding mechanical reinforcement (e.g., corner brackets, or small screws) would increase structural integrity, especially if the prototype undergoes frequent use or rough handling.

PCB Protection – The custom MOSFET board is currently exposed. Encasing it in a small enclosure or applying conformal coating would protect the solder joints and components from dust, moisture, or accidental shorts.

Repeatability for Production – The laser-cut files and wiring diagram already support repeatability. However, creating a simple assembly jig for aligning the solenoids and latch plates would ensure consistent mechanical calibration across multiple units.

With these manufacturing refinements, the prototype would become a robust, presentable product suitable for extended use or small-batch production.

Acknowledgements



Slide 4

- The toy : <https://www.youtube.com/watch?v=yyG6ifl8fPc>
- The Tablet : <https://www.amazon.co.uk/Learning-Resources-El-8897-Electronic-Maths/dp/B0C4FWC2TR>
- Wooden red toy till : <https://www.bigjigstoys.co.uk/products/shop-till?srsId=AfmBOoq-4M5Jw86TCU9FyQqlA5XqzxAYjeZ2wVG6hXA00DGLPBSzLmE>

Slide 6

- Arduino Mega image form The PiHut : https://thepihut.com/cdn/shop/files/arduino-mega-2560-rev3-arduino-a000067-40770343796931_700x.jpg?v=1691758395

Slide 7

- DFPlayerMini image form The PiHut : https://thepihut.com/cdn/shop/products/dfplayer-a-mini-mp3-player-dfrobot-dfr0299-36251812200643_800x.jpg?v=1653310079
- Pinout Image : https://wiki.dfrobot.com/dfr0299/#tech_specs

Slide 8

- DFPlayerMini wiring from Dfrobot : https://wiki.dfrobot.com/dfplayer_mini_sku_dfr0299

Slide 9

- LCD image form The PiHut : https://thepihut.com/cdn/shop/products/lcd1602-i2c-module-the-pi-hut-wav-23991-40389518819523_700x.jpg?v=1681806030
- LCD Pinout : <https://electropeak.com/learn/interfacing-i2c-16x2-character-lcd-1602-display-module-with-arduino/?srsltid=AfmBOoqqatzBbteKLTrhwTb5L-CPza7WXwfWBvP7-taIPXn2aTzZVN9I>

Slide 10

- LCD image form Core Electronics : <https://forum.core-electronics.com.au/t/waveshare-1602-lcd/18343>

Slide 11

- Arduino : <https://thepihut.com/products/arduino-mega-2560-rev3?srsltid=AfmBOopvVP02x9k6hSBHh6ACuPHe4GQpWk-mtdwM7VUcaMSYdhFGct3A>
- RFID : <https://www.az-delivery.uk/products/rfid-set>
- Solenoids : <https://www.amazon.co.uk/Electromagnetic-Solenoid-Assembly-Electirc-Cabinet/dp/B07TMWY94C>
- LCD : <https://uk.farnell.com/dfrobot/dfr0063/lcd-display-module-i2c-16x2-arduino/dp/3769973>